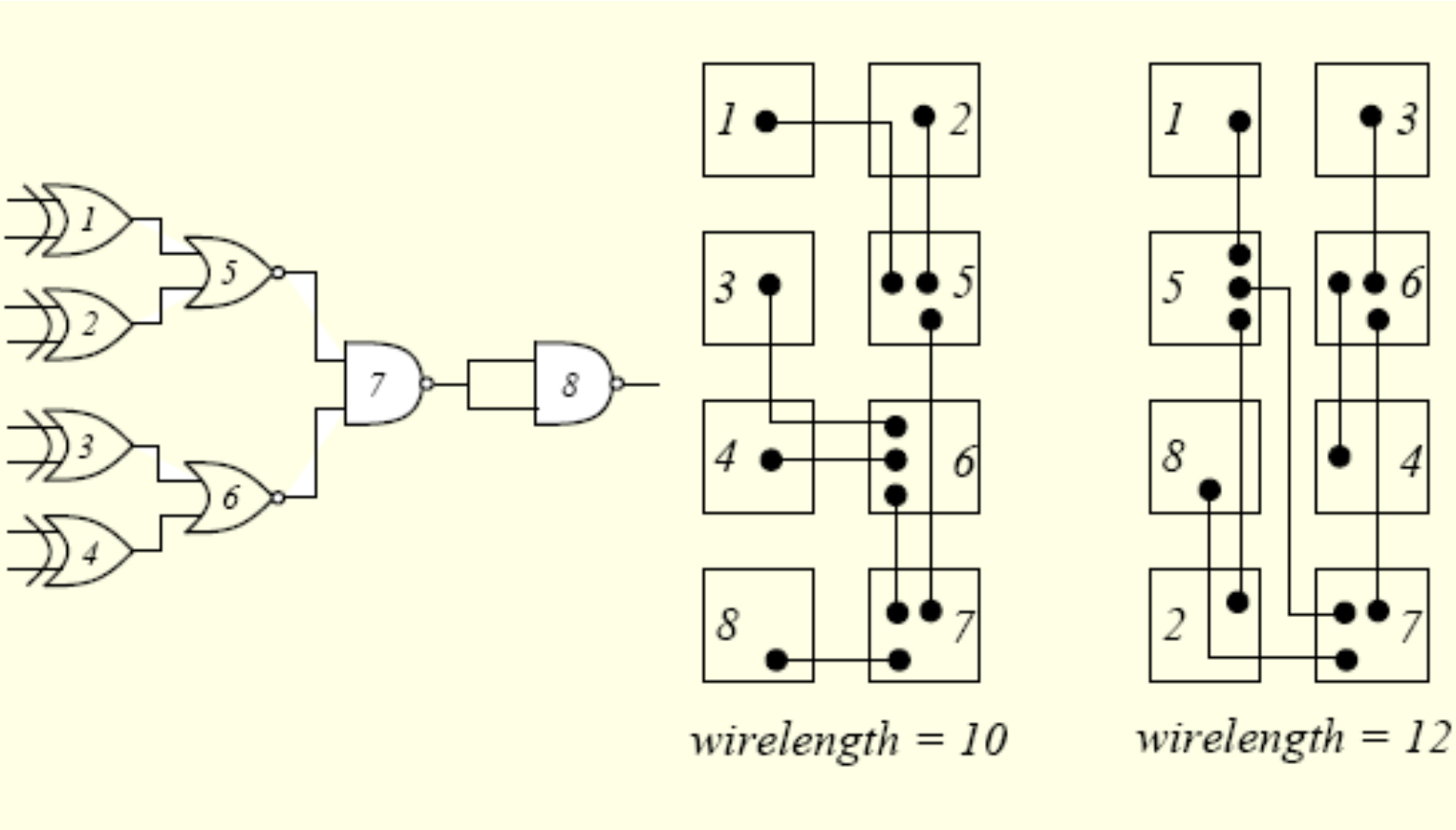
CAD for VLSI

Placement 1

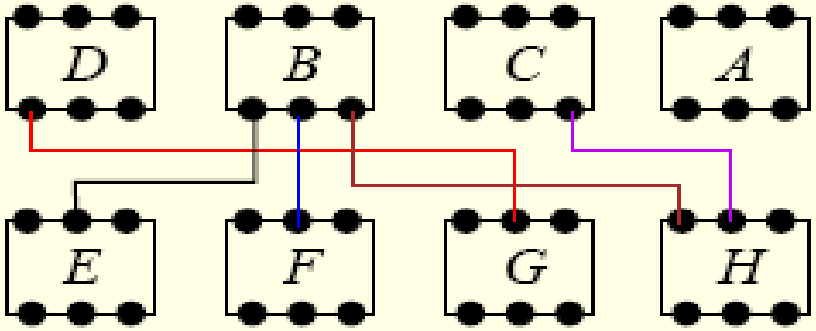
Problem formulation

- Input:
  - Blocks (standard cells and macros) B_1, ... , B_n
  - Shapes and Pin Positions for each block B_i
  - Nets N_1, ... , N_m
- Output:
  - Coordinates (x_i , y_i) for block B_i.
  - No overlaps between blocks
  - The total wire length is minimized
  - The area of the resulting block is minimized or given a fixed outline
- Other consideration: timing, routability, clock, buffering and interaction with physical synthesis

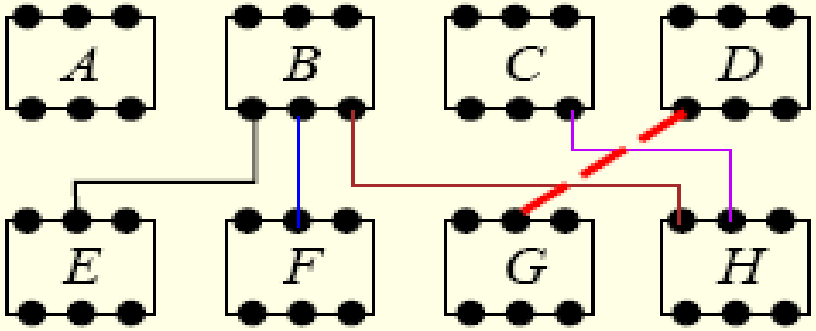
Different Wire Length



Different Routability/Chip Area



Density = 2 (2 tracks required)

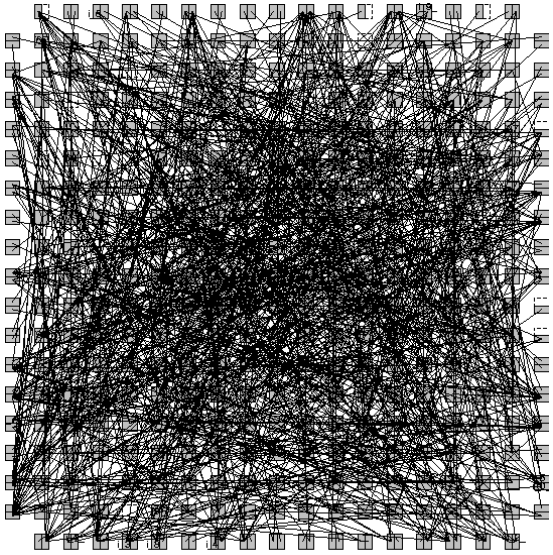


Shorter wirelength, 3 tracks required.

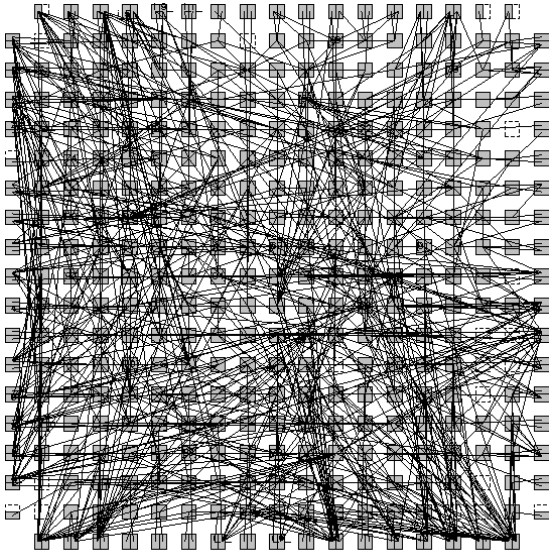
Placement can Make a Difference

- MCNC Benchmark circuit e64 (contains 230 4-LUT). Placed to a FPGA

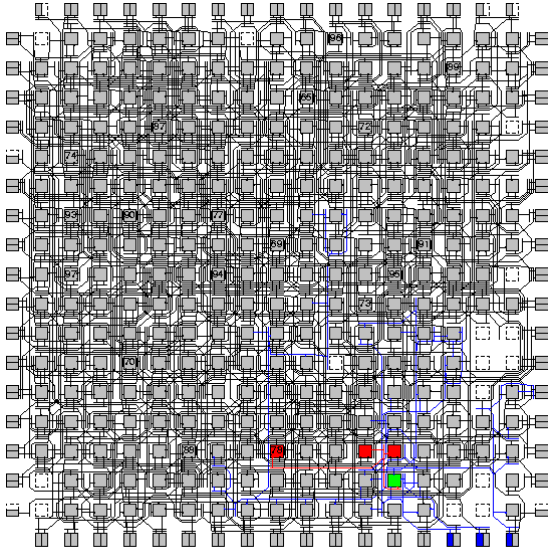
Random Initial Placement	Final Placement	After Detailed Routing



Initial Placement. Cost: 74.5592. Channel Factor: 100



Final Placement. Cost: 28.5384. Channel Factor: 100



Routing succeeded with a channel width factor of 7.

Importance of Placement

- Placement is a fundamental problem for physical design
- Glue of the physical synthesis
- Becomes very active again in recent years:
  - Many new academic placers for WL min since 2000
  - Many other publications to handle timing, routability, etc.
- Reasons:
  - Serious interconnect issues (delay, routability, noise) in deep-submicron design
    - Placement determines interconnect to the first order
    - Need placement information even in early design stages (ex: logic synthesis)
  - Placement problem becomes significantly larger
  - Cong et al. [ASPDAC-03, ISPD-03, ICCAD-03] point out that existing placers are far from optimal, not scalable, and not stable

Design Types

- ASICs
  - Lots of fixed I/Os, few macros, millions of standard cells
  - Placement densities : 40-80% (IBM)
  - Flat and hierarchical designs
- SoCs
  - Many more macro blocks, cores
  - Datapaths + control logic
  - Can have very low placement densities : < 40%
- Micro-Processor (μP) Random Logic Macros (RLM)
  - Hierarchical partitions are placement instances (5-30K)
  - High placement densities : 80%-98% (low whitespace)
  - Many fixed I/Os, relatively few standard cells

Requirements for Placers (1)

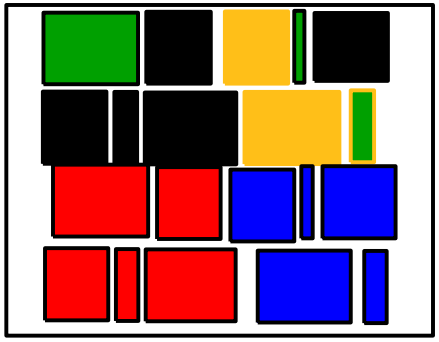
- Must handle 4-10M cells, 1000s macros
  - *64 bits* + *near-linear asymptotic complexity*
  - *Scalable/compact design database* (OpenAccess)
- Accept fixed ports/pads/pins + fixed cells
- Place macros, especially with various aspect ratios
  - Non-trivial heights and widths (ex: height=2rows)
- Honor targets and limits for net length
- Respect floorplan constraints
- Handle a wide range of placement densities (from <25% to 100% occupied), ICCAD 02

Requirements for Placers (2)

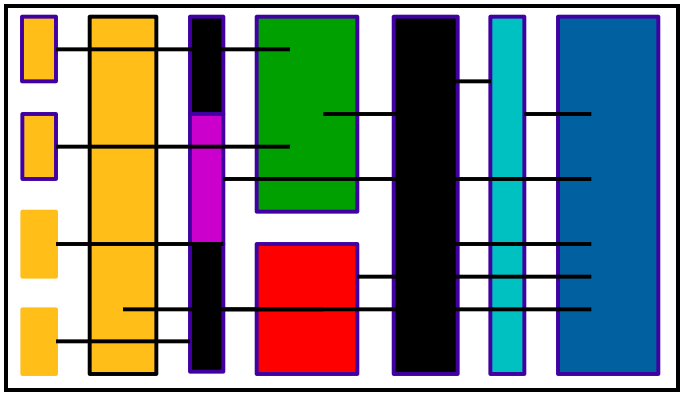
- Add / delete filler cells and Nwell contacts
- Ignore clock connections
- ECO (Engineering Change Order) placement
  - Fix overlaps after logic restructuring
  - Place a small number of unplaced blocks
- Datapath planning services
  - Ex: for cores
- Provide placement dialog services to enable cooperation across tools
  - Ex: between placement and synthesis

Placement Footprints

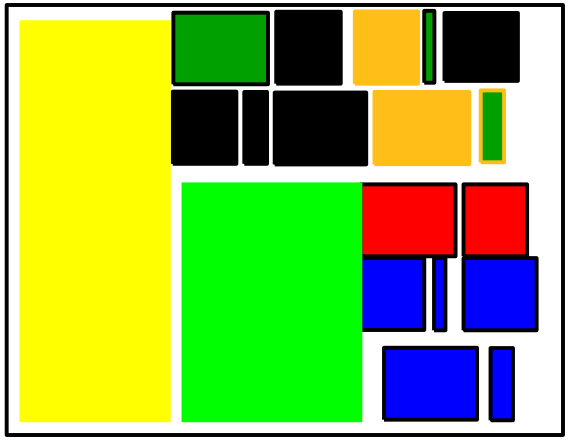
Standard Cell



Data Path

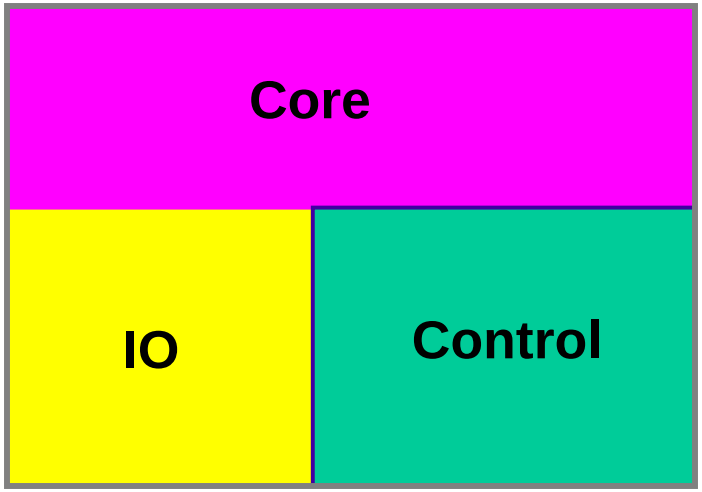


IP - Floorplanning

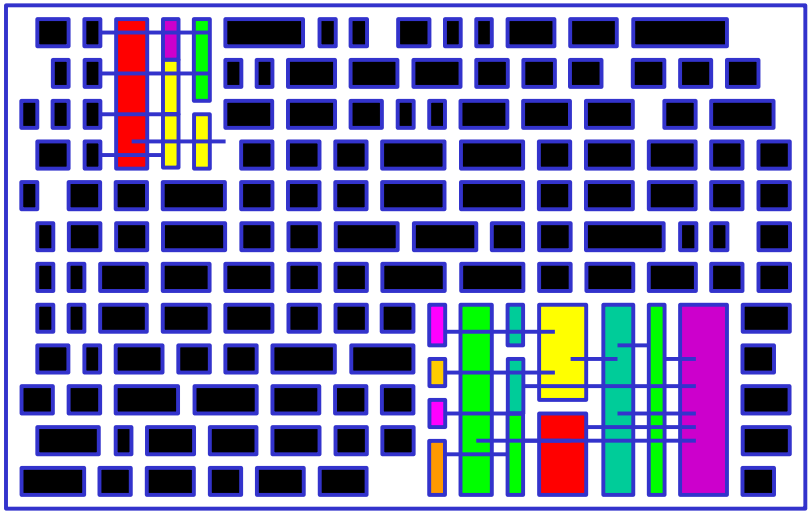


Placement Footprints

Reserved areas

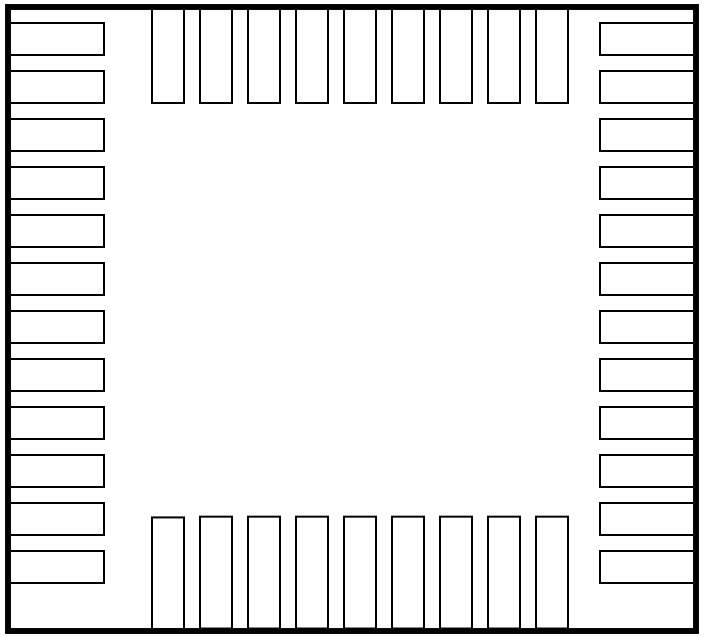


Mixed data path &
sea of gates

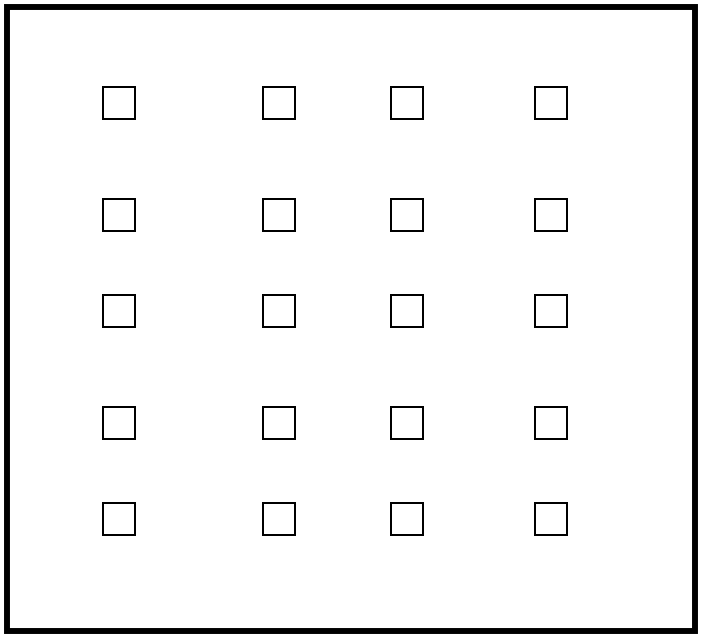


Placement Footprints

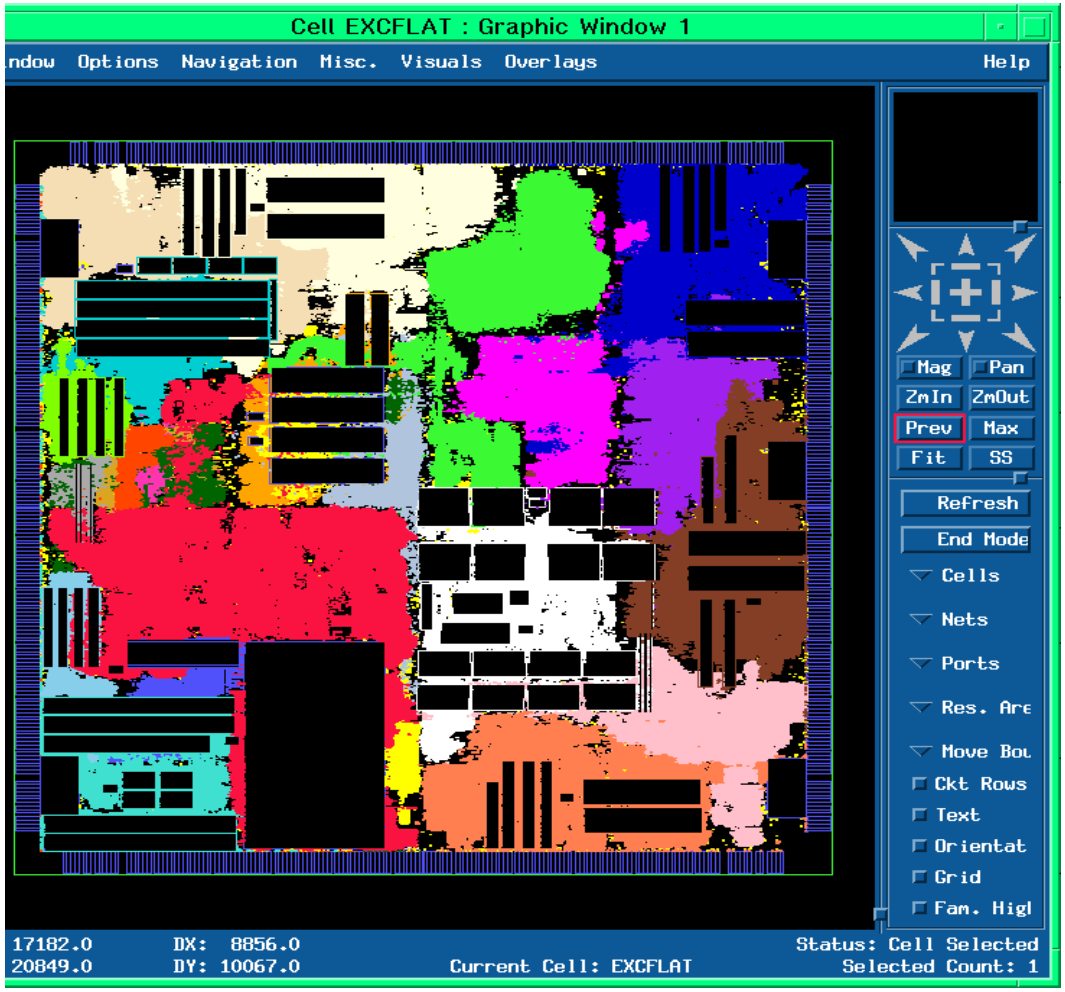
Perimeter IO



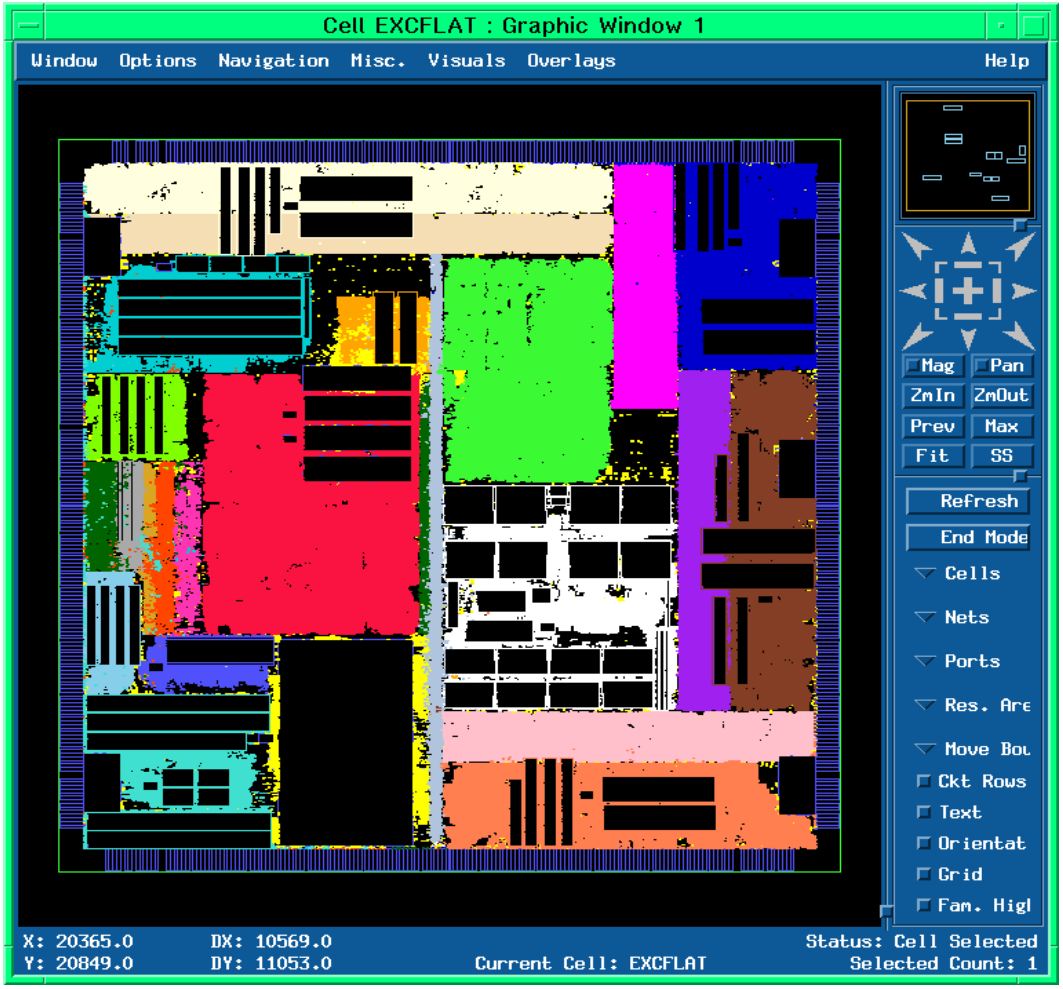
Area IO



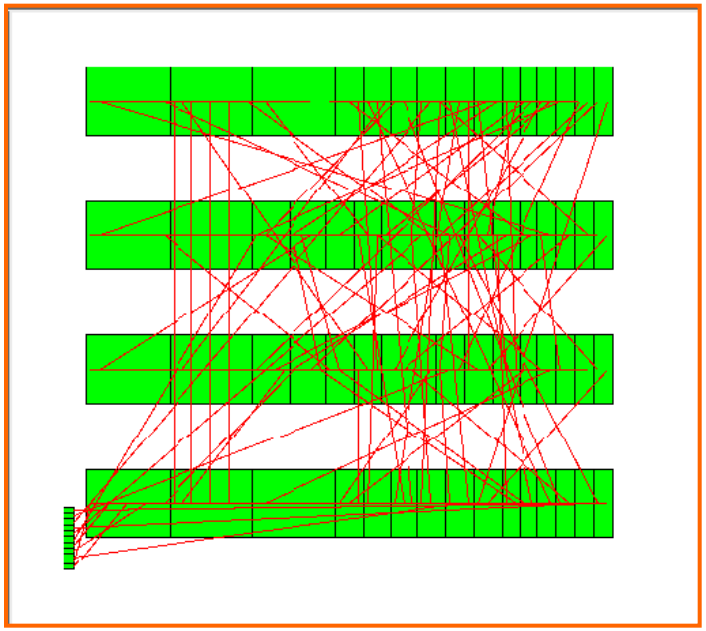
Unconstrained Placement



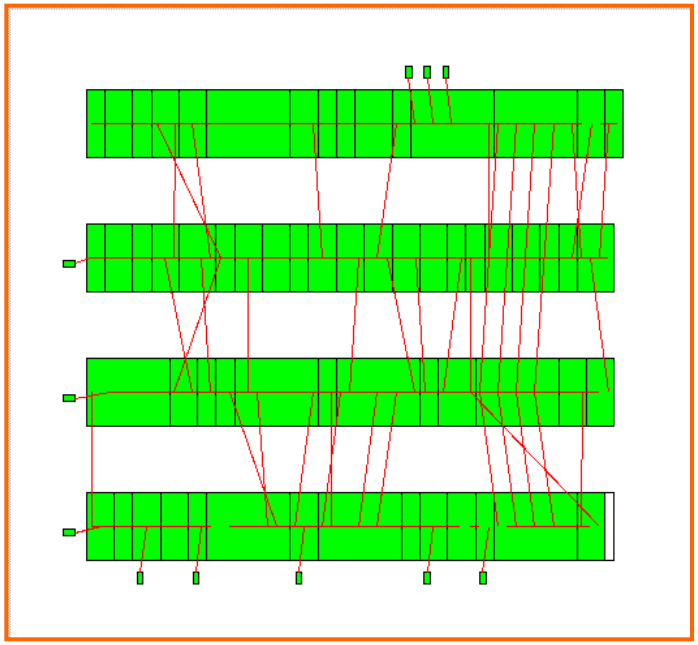
Floorplanned Placement



VLSI Global Placement Examples

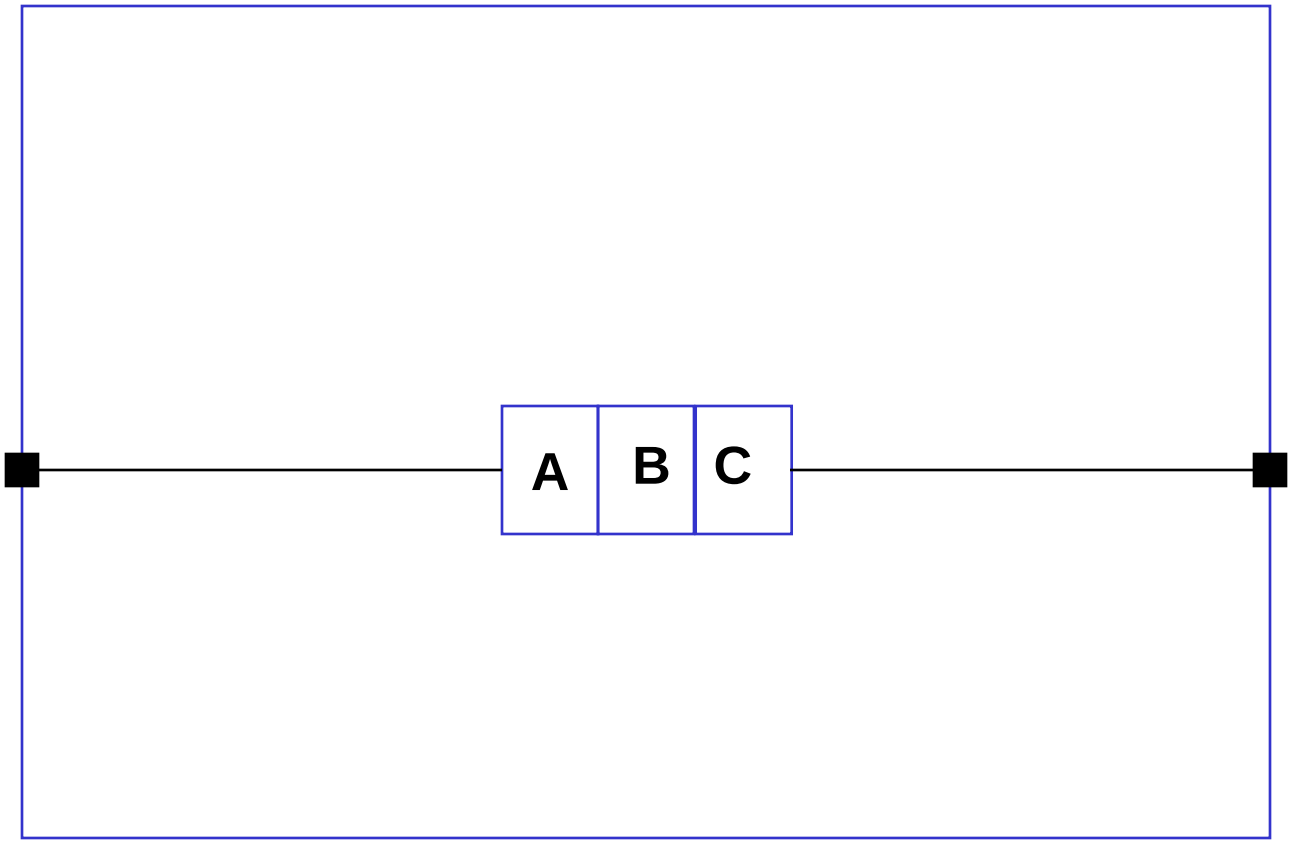


bad placement

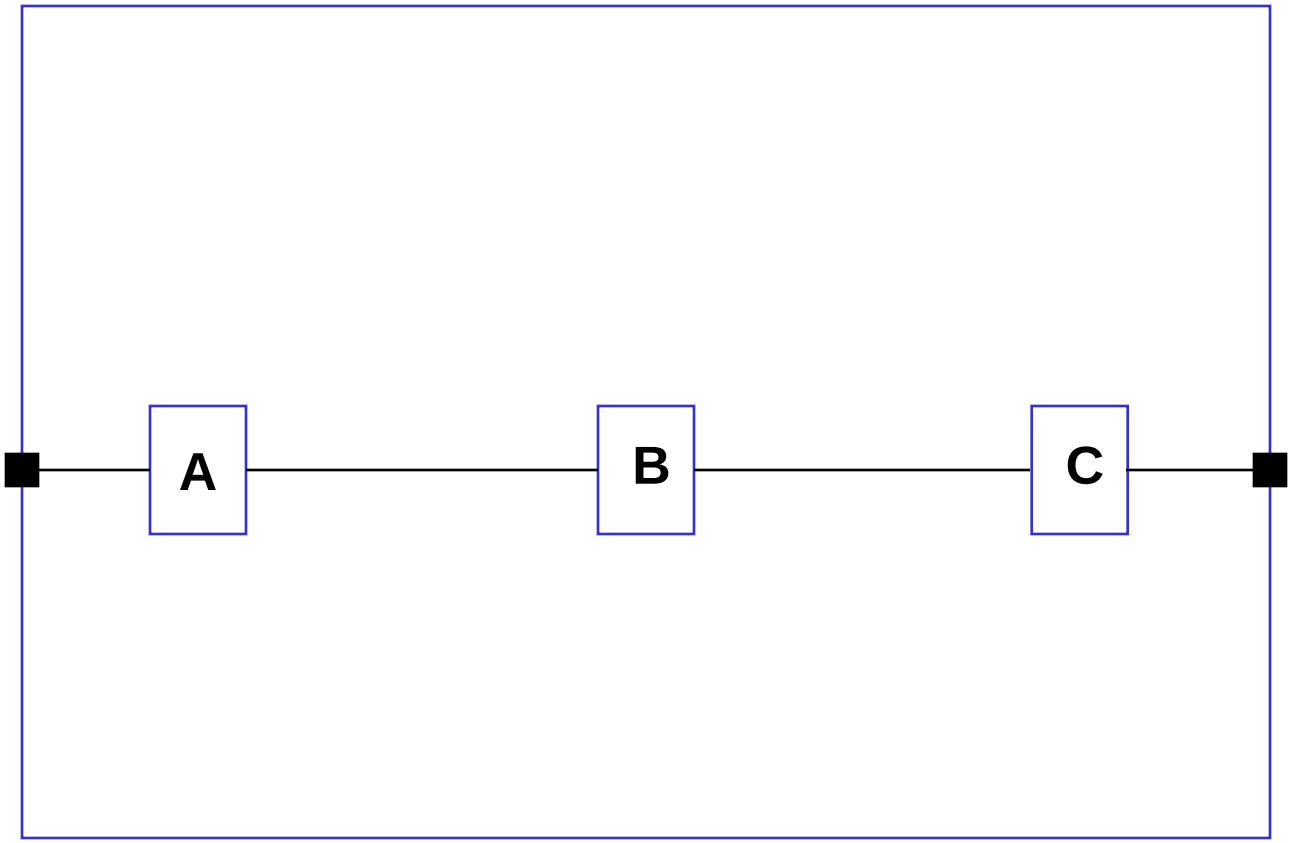


good placement

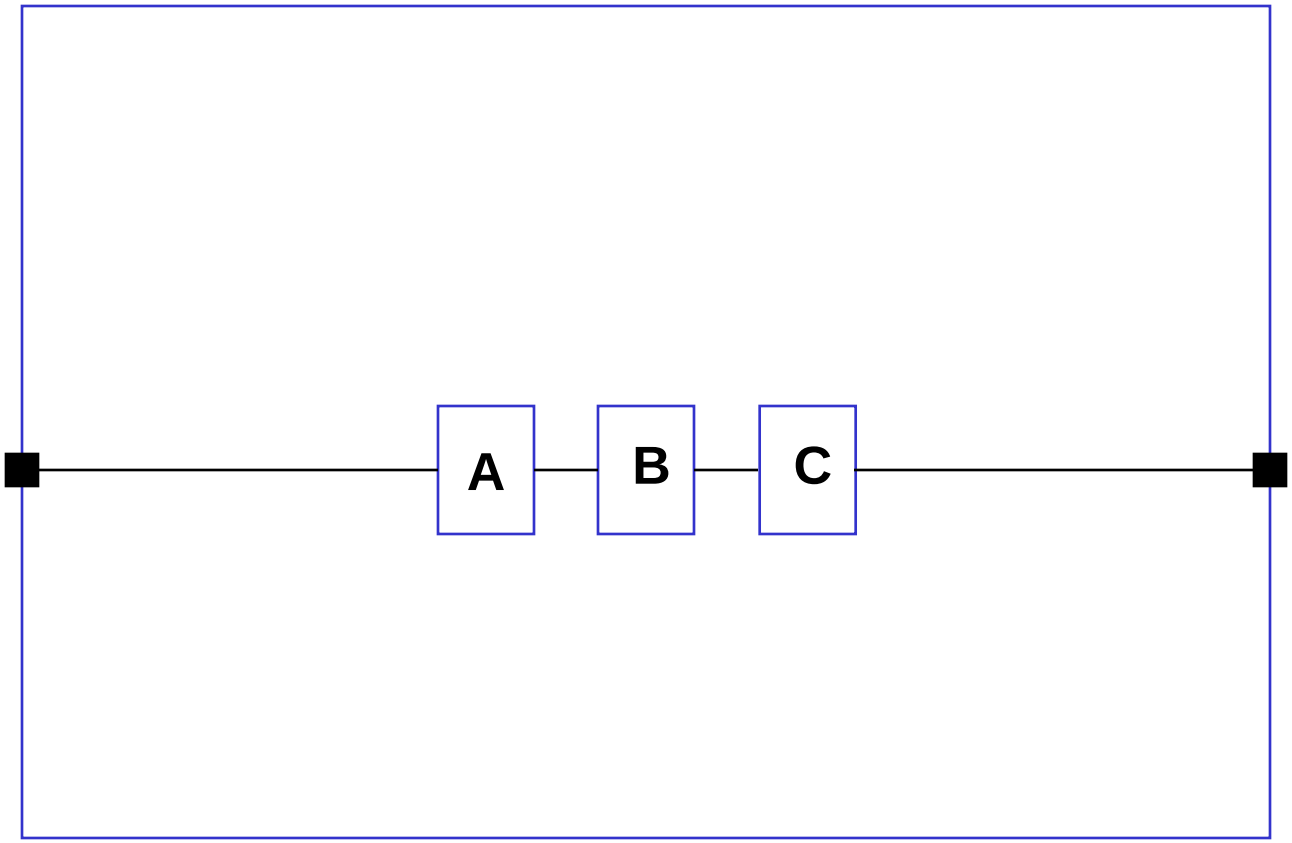
Optimal Relative Order



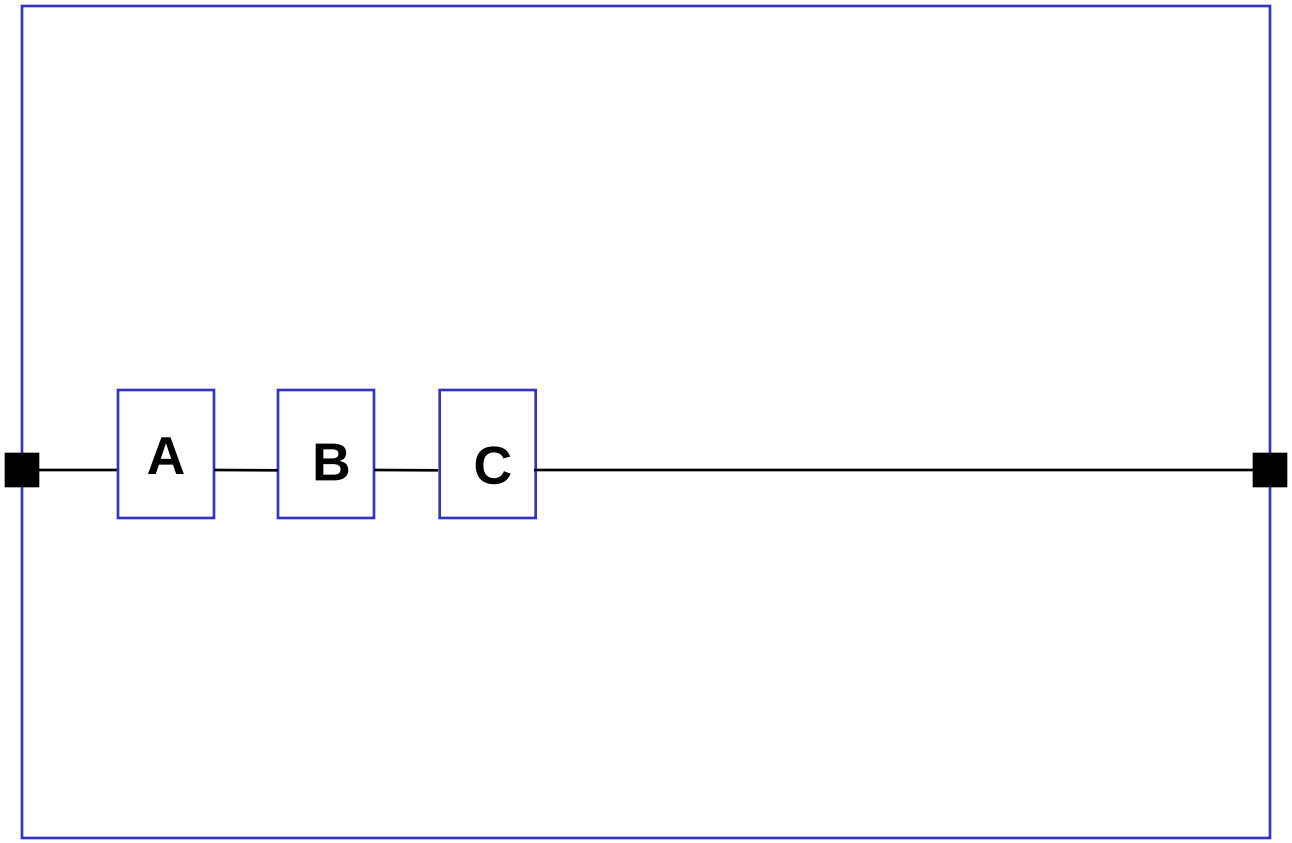
To spread ...



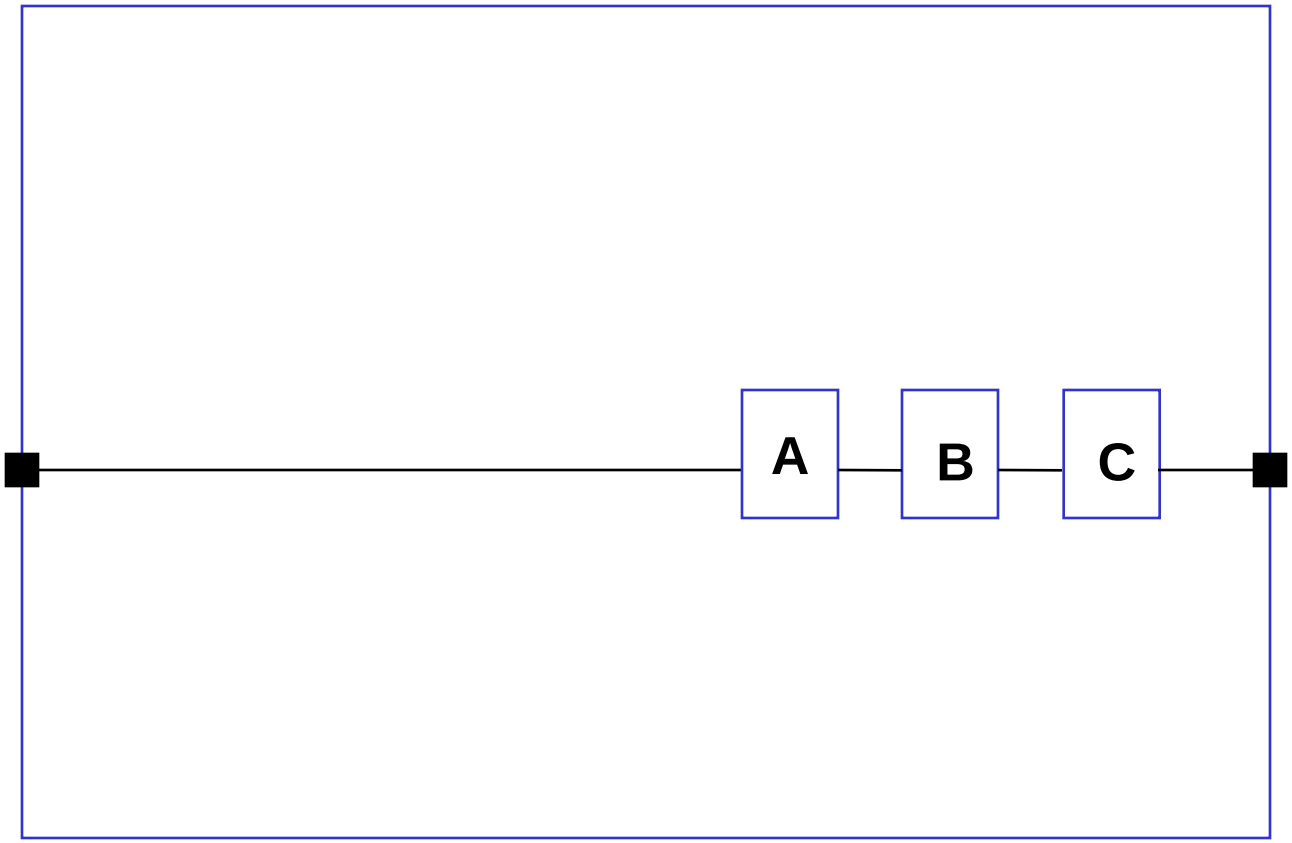
.. or not to spread



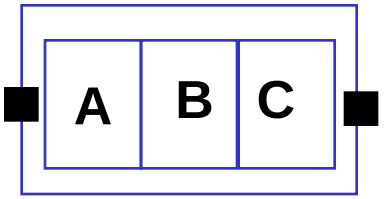
Place to the left



… or to the right



Optimal Relative Order



Without “free” space the problem is dominated by order

Major Placement Techniques

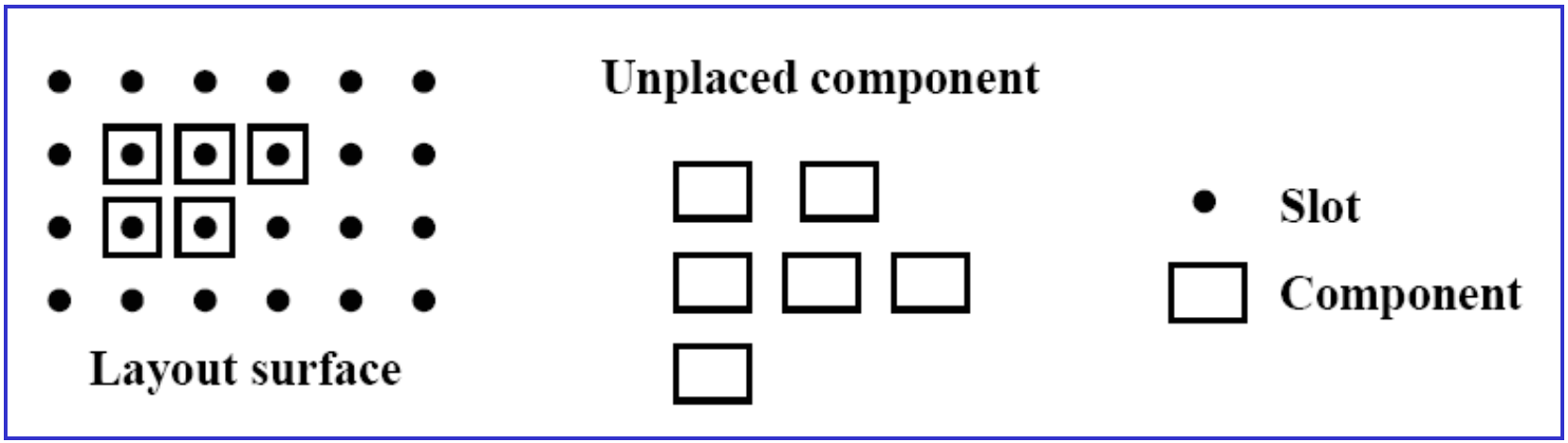
- Simulated Annealing
  - Timberwolf package [JSSC-85, DAC-86]
  - Dragon [ICCAD-00]
- Partitioning-based Placement
  - Capo [DAC-00]
  - Fengshui [DAC-2001]
- Analytical Placement
  - Gordian [TCAD-91]
  - Kraftwerk [DAC-98]
  - FastPlace [ISPD-04]
- Hall's Quadratic Placement
- Genetic Algorithm

Outline

- Wire length driven placement
- Main methods
  - Simulated Annealing
    - Gate-Array: Timberwolf package
    - Standard-Cell: Timberwolf package, Dragon
  - Partition-based methods
  - Analytical methods
  - Timing, congestion and other considerations
- Global placement (rough location)
- Detailed placement (legalization)

A down-to-the-earth method

- Clustering (Select unplaced components and place them)
  - SELECT: choose unplaced component that is most strongly connected to all (or any single) of the placed components
  - PLACE: place the selected component at a slot such that a certain “cost” of the partial placement is minimized
  - Simple and fast: ideal for initial placement



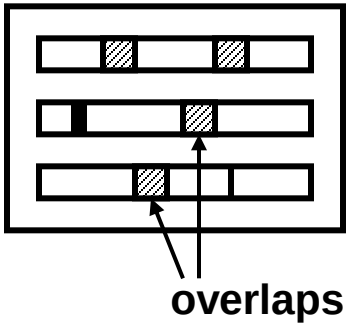
Simulated Annealing Based Placement

- *"The Timberwolf Placement and Routing Package", Sechen, Sangiovanni; IEEE Journal of Solid-State Circuits, vol SC-20, No. 2(1985) 510-522*
- *"Timber wolf 3.2: A New Standard Cell Placement and Global Routing Package" Sechen, Sangiovanni, 23rd DAC, 1986, 432-439*

- Timberwolf
  - Stage 1
    - Modules are moved between different rows as well as within the same row
    - modules overlaps are allowed
    - when the temperature is reduced below a certain value, stage 2 begins
  - Stage 2
    - Remove overlaps
    - Annealing process continues, but only interchanges adjacent modules within the same row

Solution Space

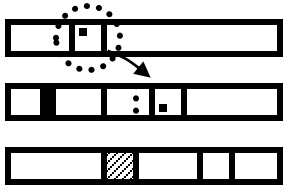
All possible arrangements of modules into rows possibly with overlaps



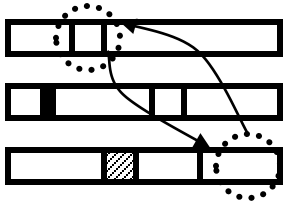
Neighboring Solutions

Three types of moves:

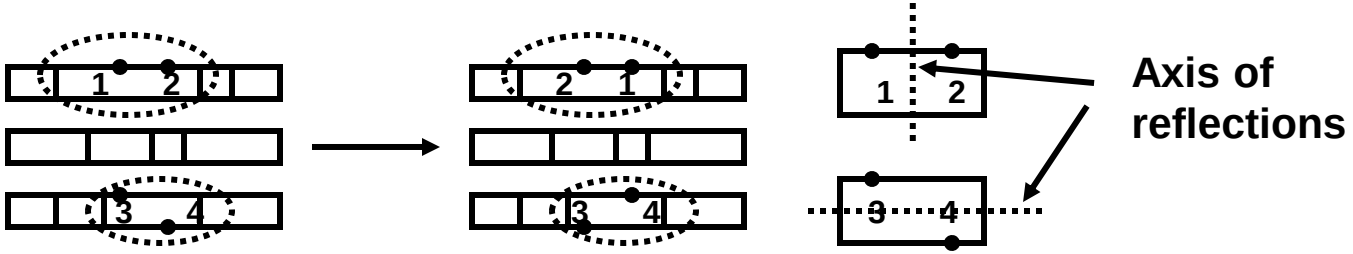
M1: Displace a module to a new location



M2: Interchange two modules



M3: Change the orientation of a module (flip)



Move Selection

- Timberwolf first try to select a move between M1 and M2
  - Prob(M1)=4/5
  - Prob(M2)=1/5

M1: Displacement

M2: Interchange

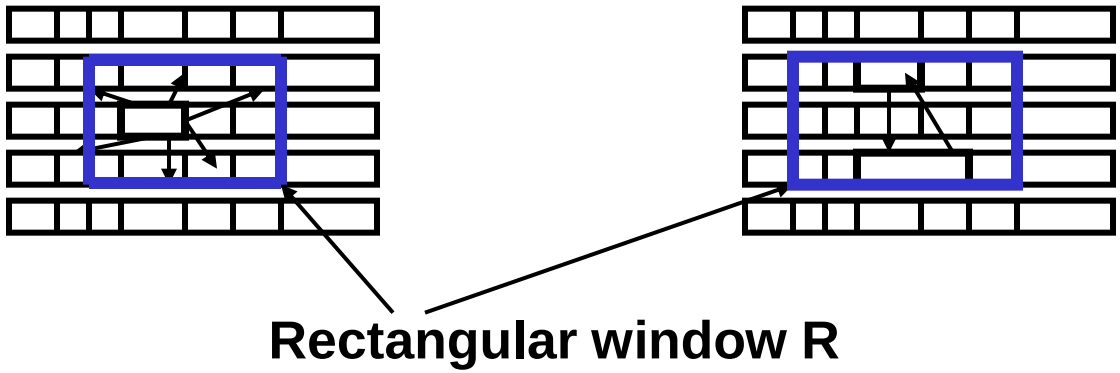
M3: Reflection

- If a move of type M1 is chosen (for certain module) and it is rejected, then a move of type M3 (for the same module) will be chosen with probability 1/10

- Restriction on
  - How far a module can be displaced
  - What pairs of modules can be interchanged

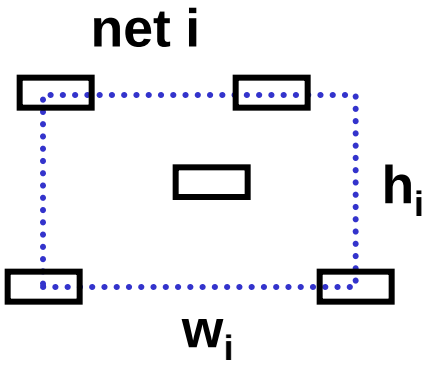
Move Restriction

- Range Limiter
  - At the beginning, R is very large, big enough to contain the whole chip
  - Window size shrinks slowly as the temperature decreases. In fact, height and width of R $\propto$ log(T)
  - Stage 2 begins when window size are so small that no inter-row modules interchange is permitted



Cost Function

$\Psi = C_1+C_2+C_3$



C_1: $\sum_i (\alpha_i w_i + \beta_i h_i)$

α_i, β_i are horizontal and vertical weights, respectively

α_i =1, β_i =1 $\Rightarrow$1/2 •perimeter of bounding box

- Critical nets: Increase both α_i and β_i
- Preferred metal layer routing: if vertical wirings are "cheaper" than horizontal wirings, we can use smaller vertical weights, i.e. $\beta_i < \alpha_i$

Cost Function (Cont'd)

C_2: Penalty function for module overlaps

O(i,j) = amount of overlaps in the X-dimension between modules i and j

$$C_2 = \sum_{i \neq j} \left(O(i,j) + \alpha \right)^2$$

α — offset parameter to ensure $C_2 \rightarrow 0$ when $T \rightarrow 0$

C_3: Penalty function that controls the row lengths

Desired row length = d(r)

l(r) = sum of the widths of the modules in row r

$$C_3 = \sum_r \beta \left| l(r) - d(r) \right|$$

Annealing Schedule

- $T_k = r(k) \bullet T_{k-1}$ k= 1, 2, 3, ….
  - r(k) increase from 0.8 to max value 0.94 and then decrease to 0.1
- At each temperature, a total number of K•n attempts is made
  - n: number of modules
  - K: user specified constant

Dragon2000: Standard-Cell Placement Tool for Large Industry Circuits

M. Wang, X. Yang, and M. Sarrafzadeh,

ICCAD-2000

pages 260-263

Main Idea

- Simulated annealing based
  - 1.9x faster than iTools 1.4.0 (commerical version of TimberWolf)
  - Comparable wirelength to iTools (i.e., very good)
  - Performs better for larger circuits
  - Still very slow compared with the other approaches
  - Also shown to have good routability
- Top-down hierarchical approach
  - hMetis to recursively quadrisect into 4^h bins at level h
  - Swapping of bins at each level by SA to minimize WL
  - Terminates when each bin contains < 7 cells
  - Then swap single cells locally to further minimize WL
- Detailed placement is done by greedy algorithm

Outline

- Wire length driven placement
- Main methods
  - Simulated Annealing
  - Partition-based methods
    - Capo
    - Feng-shui
  - Analytical methods
  - Timing, congestion and other considerations
- Global placement (rough location)
- Detailed placement (legalization)

Partition-based methods

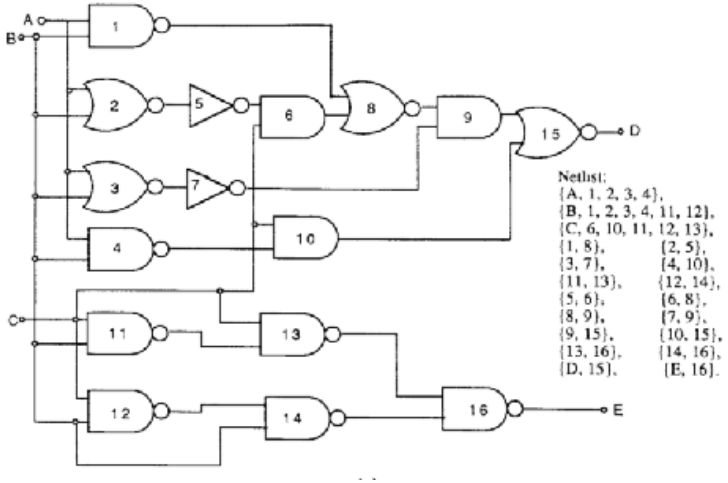
- Partitioning methods
  - FM
  - Multilevel techniques, ex: hMetis
- Two academic open source placement tools
  - Capo (UCLA/UCSD/Michigan): multilevel FM
  - Feng-shui (SUNY Binghamton): use hMetis
- Pros and cons
  - Fast
  - Not stable

Partitioning-based Approach

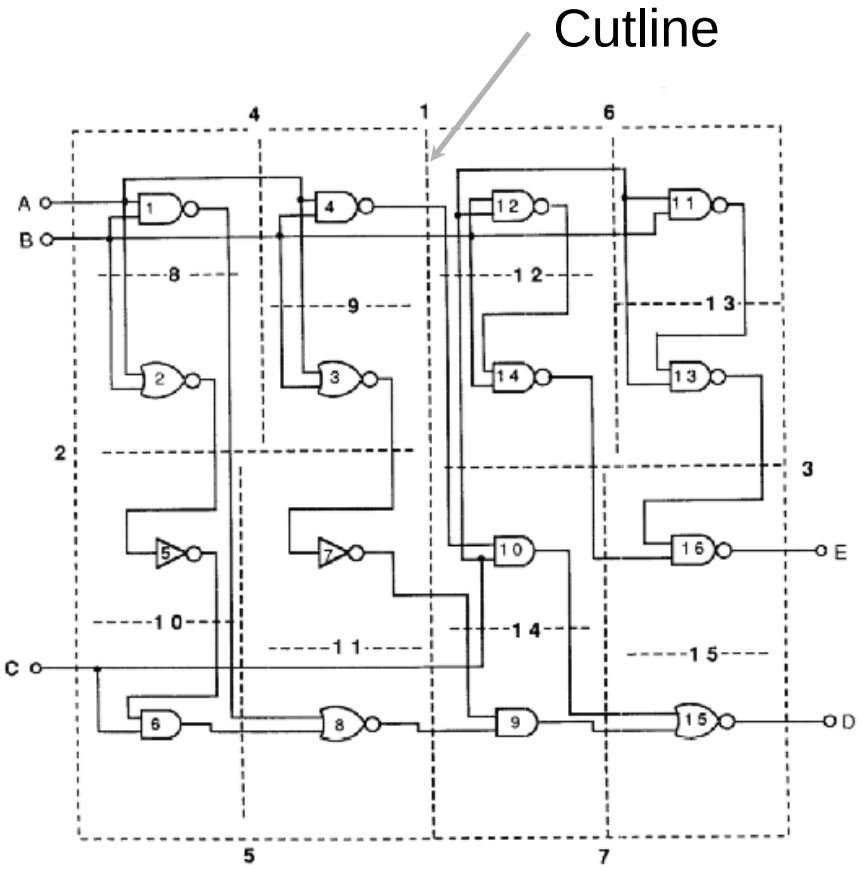
- Try to group closely connected modules together
- Repetitively divide a circuit into sub-circuits such that the cut value is minimized
- Also, the placement region is partitioned (by cutlines) accordingly
- Each sub-circuit is assigned to one partition of the placement region

Note: Also called min-cut placement approach

An Example



Circuit



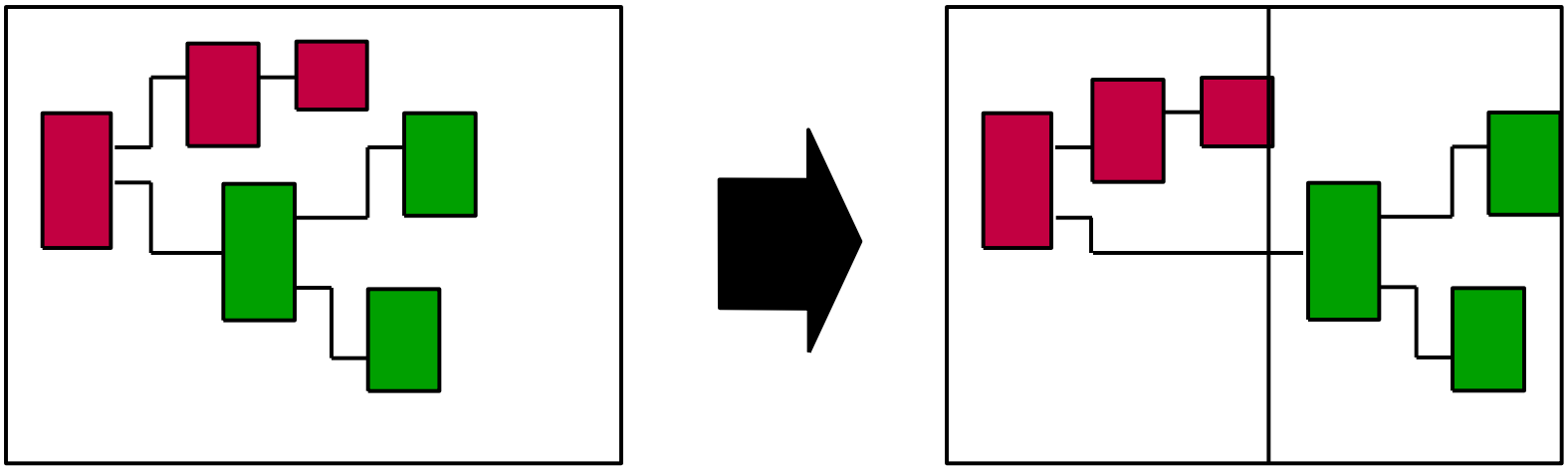
Placement

Variations

- There are many variations in the partitioning-based approach. They are different in:
  - The objective function used
  - The partitioning algorithm used
  - The selection of cutlines

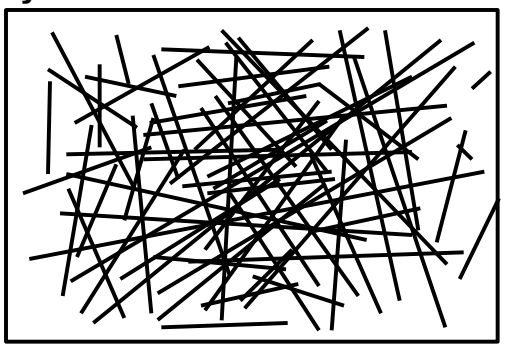
Recap of Partitioning

- Objective:
  - Given a set of interconnected blocks, produce two sets that are of equal size, and such that the number of nets connecting the two sets is minimized.

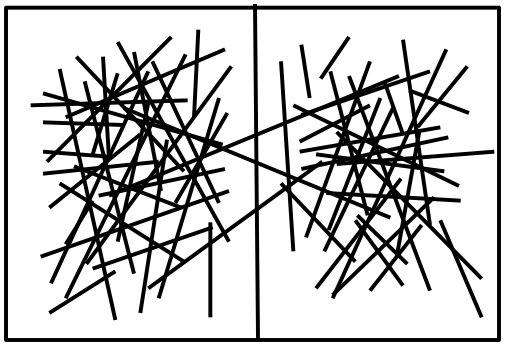


Algorithm

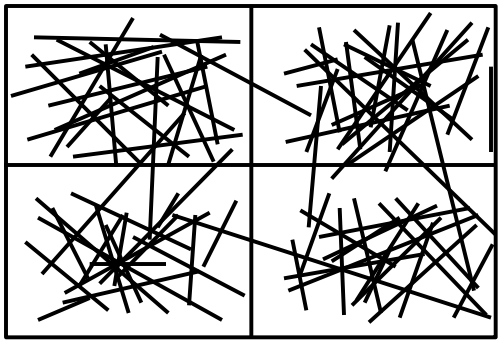
```
list_of_sets = entire_chip;
while(any_set_has_2_or_more_objects(list_of_sets))
{
        for_each_set_in(list_of_sets)
        {
              partition_it();
        }
        /* each time through this loop, the number of   */
        /* sets in the list doubles.                    */
}
```



Initial Random Placement



After Cut 1



After Cut 2

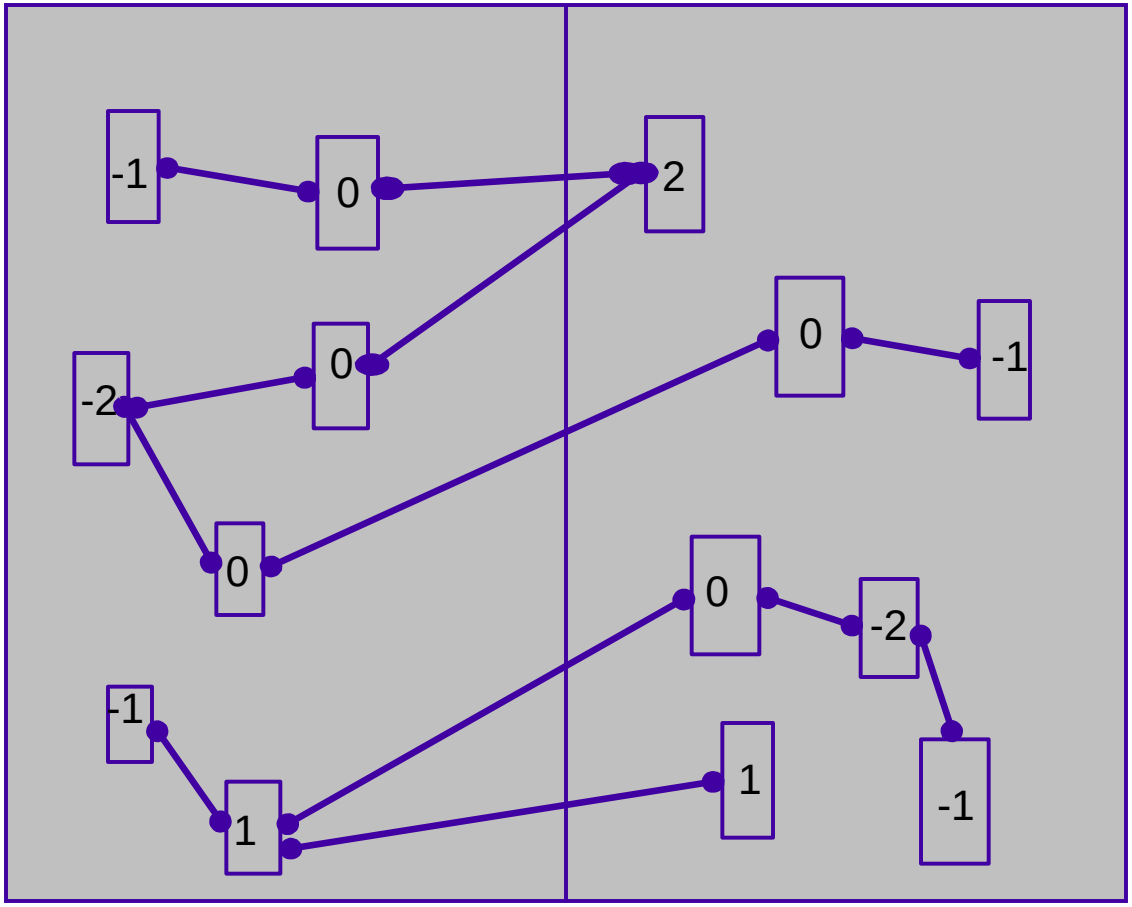
FM Partitioning

- Moves are made based on object gain
- Object Gain: The amount of change in cut crossings that will occur if an object is moved from its current partition into the other partition
- A pass description

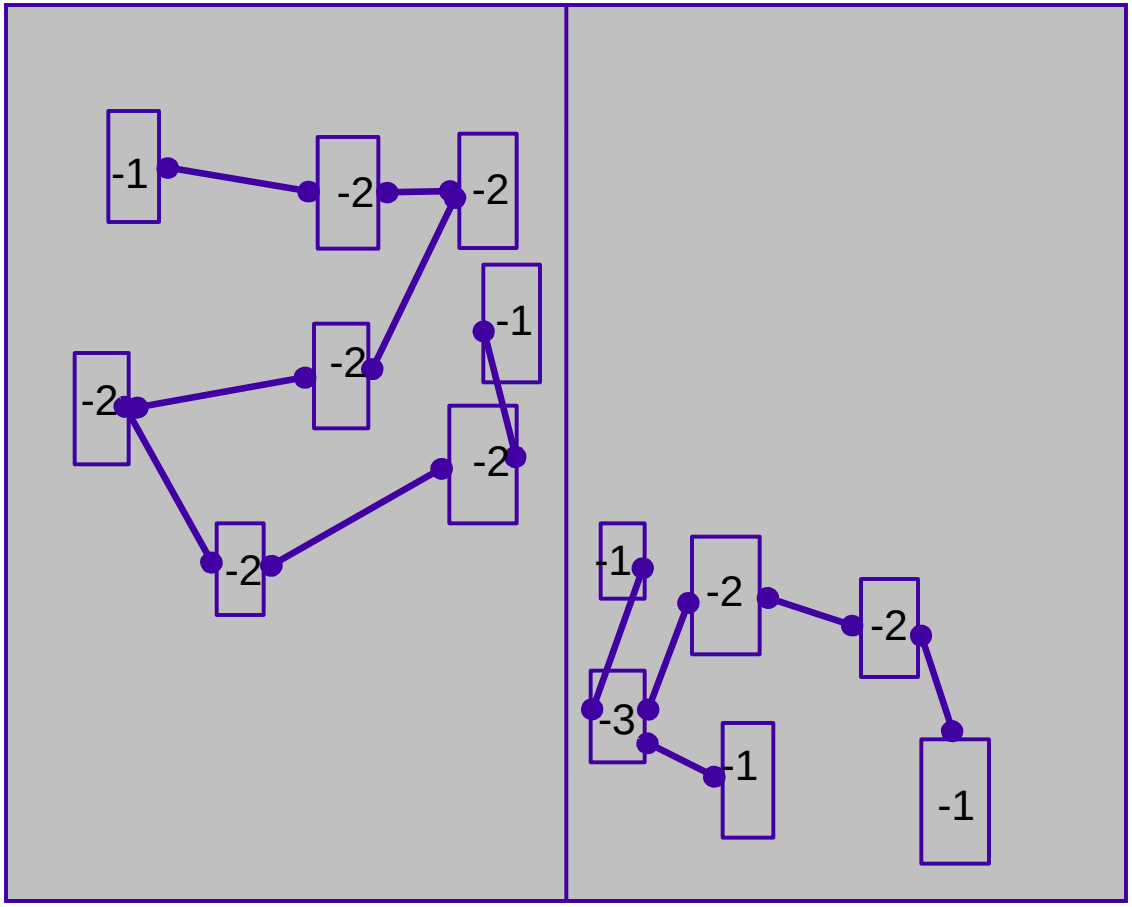
  While there is unlocked object

  1. Each object is assigned a gain
  2. Objects are put into a sorted gain list
  3. The object with the highest gain from the larger of the two sides is selected and moved
  4. The moved object is "locked"
  5. Gains of "touched" objects are recomputed
  6. Gain lists are resorted
- Repeat the pass until there is no improvement

FM Partitioning

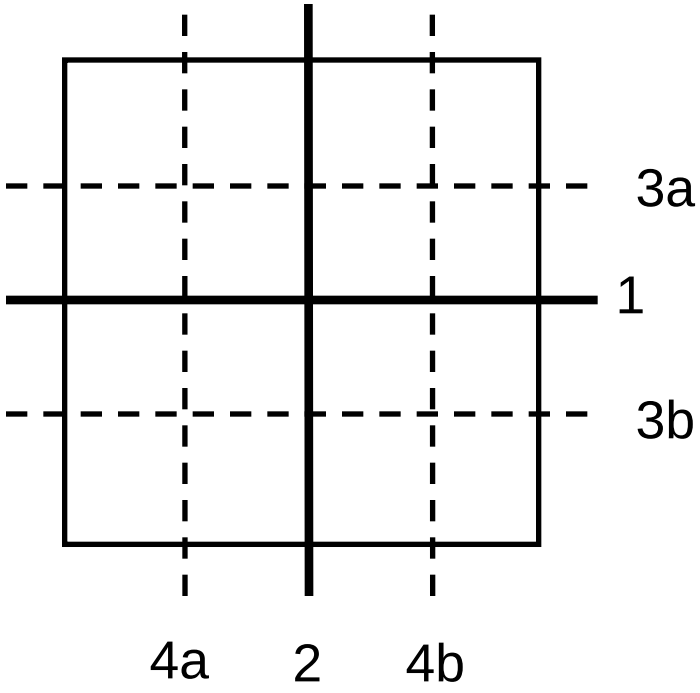


FM Partitioning



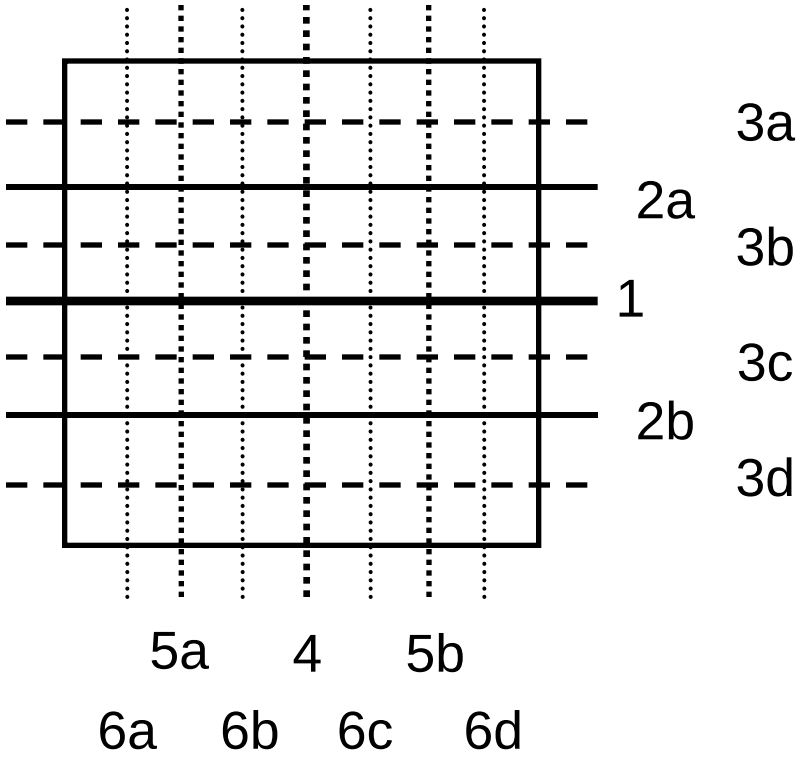
Quadrature Placement Procedure

- Very suitable for circuits with high routing density in the center



Bisection Placement Procedure

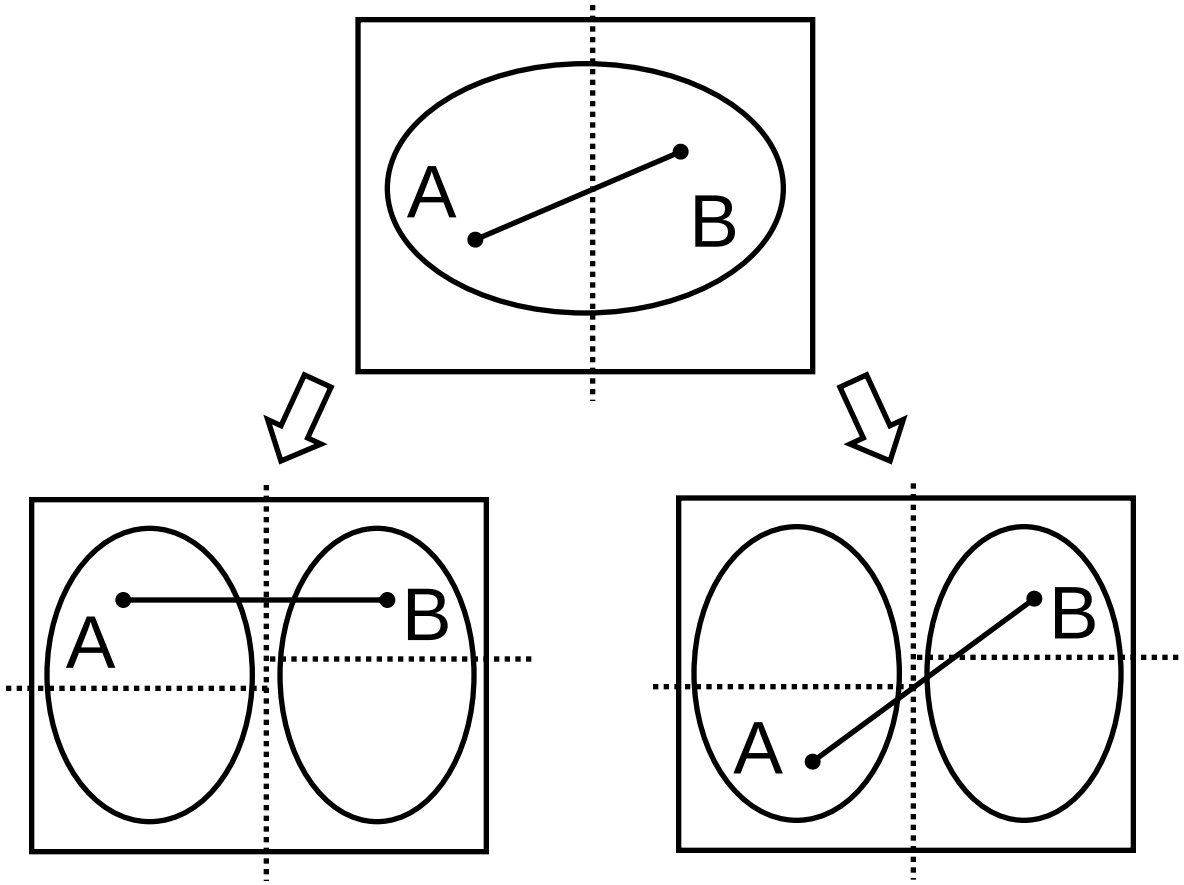
- Good for standard-cell placement



Terminal Propagation Algorithm by Dunlop and Kernighan

"A Procedure for Placement of Standard-Cell VLSI Circuits", TCAD, 4(1):92-98, Jan. 1985.

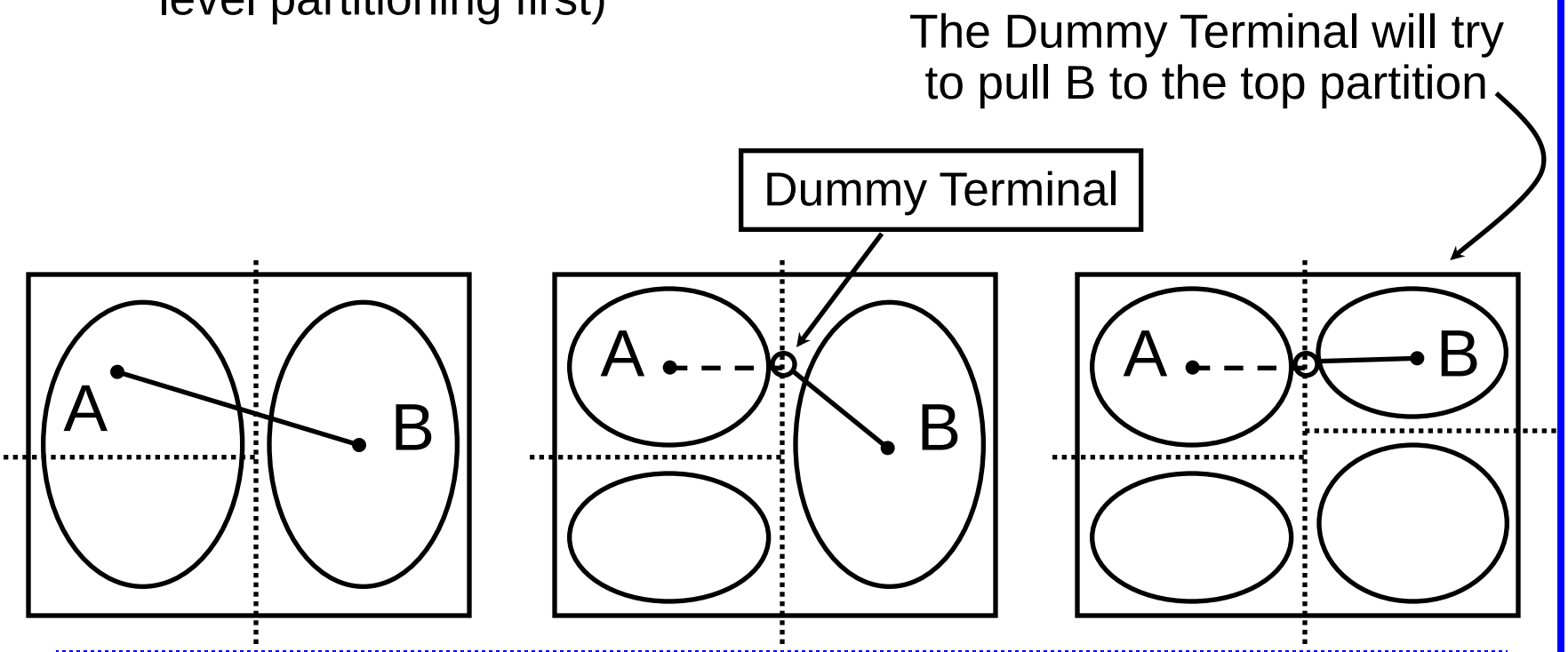
Problem of Partitioning Subcircuits



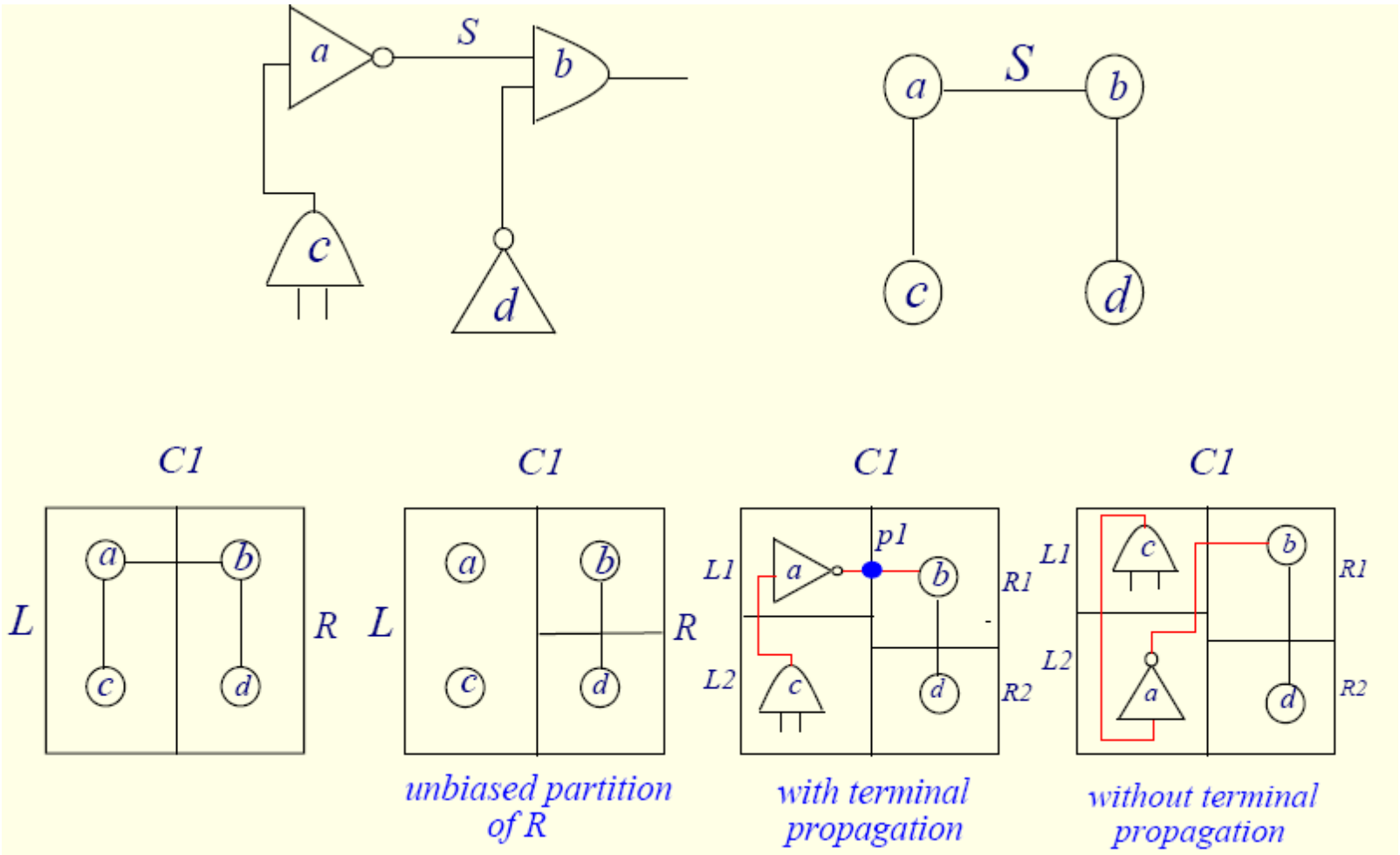
Cost of these 2 partitionings are not the same

Terminal Propagation

- Need to consider nets connecting to external terminals or other modules as well
- Do partitioning in a breadth-first manner (i.e., finish all higher-level partitioning first)

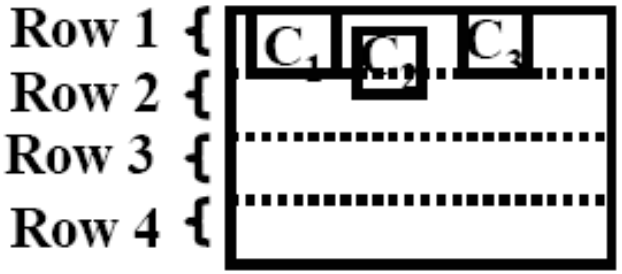


Terminal Propagation



Creating Circuit Rows

- Terminal propagation reduce overall area by ~30%
- Creating rows
  - Choose α and β preferably to balance row length (during re-arrangement)

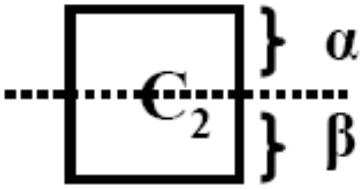


cells in $C_1 \rightarrow$ row1

cells in $C_3 \rightarrow$ row1

cells in C_2

Row 1 Row 2



$\alpha + \beta = 1$

Can Recursive Bisection Alone Produce Routable Placement? (Name of placer: Capo)

Andrew Caldwell, Andrew Kahng, and Igor Markov

DAC-2000

Capo Overview

- Standard cell placement, Fixed-die context
- Pure recursive bisectioning placer
  - Several minor techniques to produce good bisections
- Produce good results mainly because:
  - Improvement in mincut bisection using multi-level idea in the past few years
  - Pay attention to details in implementation
- Implementation with good interface (LEF/DEF and GSRC bookshelf) available on web

Capo Approach

- Recursive bisection framework:
  - Multi-level FM for instances with >200 cells
  - Flat FM for instances with 35-200 cells
  - Branch-and-bound for instances with <35 cells
- Careful handling partitioning tolerance:
  - Uncorking: Prevent large cells from blocking smaller cells to move
  - Repartitioning: Several FM calls with decreasing tolerance
  - Block splitting heuristics: Higher tolerance for vertical cut
  - Hierarchical tolerance computation: Instance with more whitespace can have a bigger partitioning tolerance

Partitioning-based Approach

- Pros:
  - Very fast
  - Great quality
  - Scales nearly linearly with problem size

- Cons:
  - Non-trivial to implement
  - Very directed algorithm, but this limits the ability to deal with miscellaneous constraints
  - Not stable (if there is minor change)

Summary for Partition-based Placement

- Improvement in mincut partitioning are conducive to better wirelength and congestion
- Routable placements can be produced in most cases without explicit congestion management
  - Explicit congestion control may still be useful in some cases
- Better weighted wirelength often implies better routed wirelength, but not always

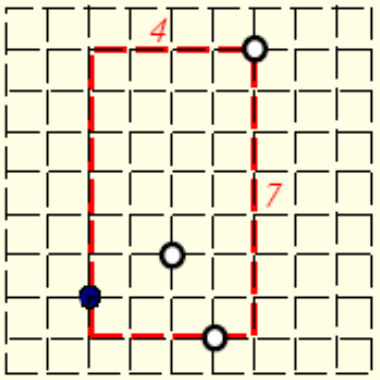
Outline

- Wire length driven placement
- Main methods
  - Simulated Annealing
  - Partition-based methods
  - Analytical methods
    - Quadratic placement with Gordian-type method
    - Force-directed method
  - Timing, congestion and other considerations
- Global placement (rough location)
- Detailed placement (legalization)

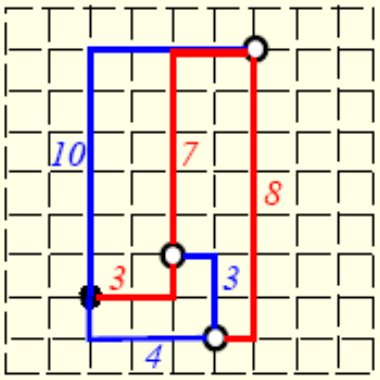
Wire Length Estimation Methods

- **Semi-perimeter method:** Half the perimeter of the bounding rectangle that encloses all the pins of the net to be connected. Most widely used approximation!
- **Complete graph:** Since #edges in a complete graph ($\frac{n(n-1)}{2}$) is $\frac{n}{2}\times$ # of tree edges $(n-1)$, $wirelength \approx \frac{2}{n}\sum_{(i,j)\in net} dist(i,j)$.
- **Minimum chain:** Start from one vertex and connect to the closest one, and then to the next closest, etc.
- **Source-to-sink connection:** Connect one pin to all other pins of the net. Not accurate for uncongested chips.
- **Steiner-tree approximation:** Computationally expensive.
- **Minimum spanning tree**

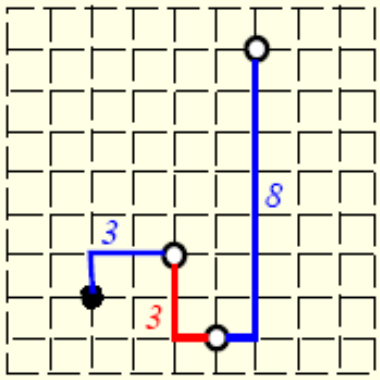
Wire Length Estimation Methods



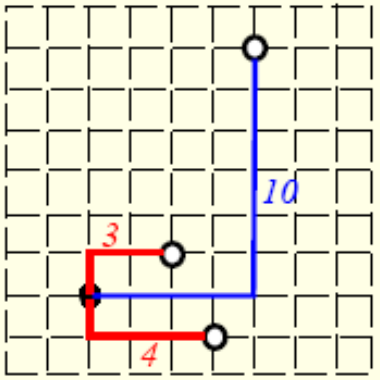
semi−perimeter len = 11



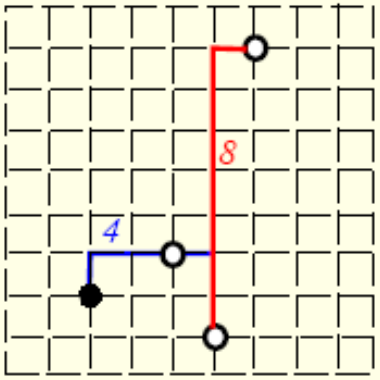
*complete graph len * 2/n = 17.5*



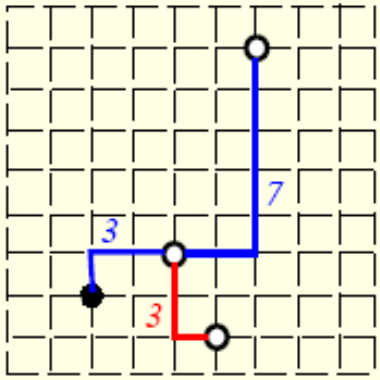
chain len = 14



source−to−sink len = 17



Steiner tree len = 12

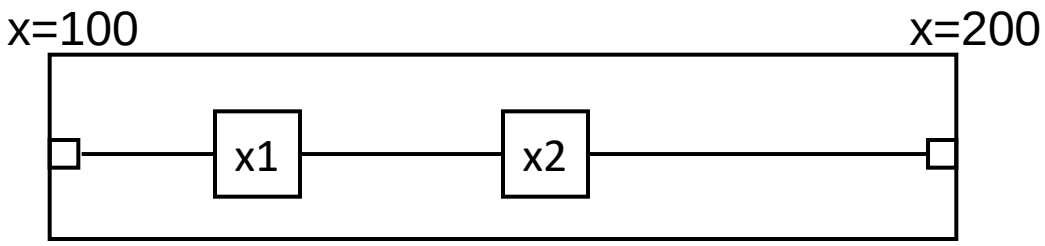


Spanning tree len = 13

Analytical Placement

- Write down the placement problem as an analytical mathematical problem
- Solve the placement problem directly
- Example:
  - Sum of squared wire length is quadratic in the cell coordinates.
  - So the wire length minimization problem can be formulated as a quadratic program.
  - It can be proved that the quadratic program is convex, hence polynomial time solvable

Toy Example

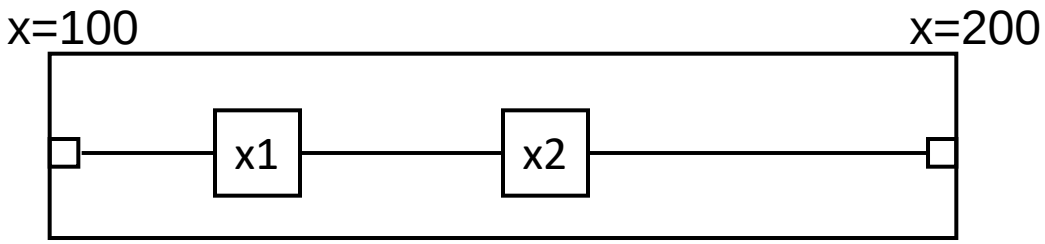


$Cost =$

$\frac{d}{dx_1} Cost =$

$\frac{d}{dx_2} Cost =$

Toy Example



setting the partial derivatives = 0 we solve for the minimum Cost:

Interpretation of matrices A and B:
The diagonal values A[i,i] correspond to the number of connections to xi
The off diagonal values A[i,j] are -1 if object i is connected to object j, 0 otherwise
The values B[i] correspond to the sum of the locations of fixed objects connected to object i

Why formulate the problem this way?

- Because we can
- Because it is trivial to solve
- Because there is only one solution
- Because the solution is a global optimum
- Because the solution conveys “relative order” information
- Because the solution conveys “global position” information

Gordian: A Quadratic Placement Approach

- Global optimization:
  - solves a sequence of quadratic programming problems
- Partitioning:
  - enforces the non-overlap constraints

- *Ref. 1: "Gordian: VLSI Placement by Quadratic Programming and slicing Optimization", by J. M. Kleinhans, G.Sigl, F.M. Johannes, K.J. Antreich IEEE Trans. On CAD, March 1991. pp 356-365*
- *Ref. 2: "Analytical Placement: A Linear or a Quadratic Objective Function?" By G. Sigl, K. Doll, F.M. Johannes, DAC'91 pp 427-423*

Quadratic Placement Formulation

Let (x_i, y_i) = Coordinates of the center of cell i

w_{ij} = Weight of the net between cell i and cell j

$\mathbf{x}, \mathbf{y}$ = Solution vectors

Cost of the net between cell i and cell j

$$= \frac{1}{2} w_{ij} \left((x_i - x_j)^2 + (y_i - y_j)^2 \right)$$

$$\text{Total cost} = \frac{1}{2} \mathbf{x}^T Q \mathbf{x} + d_x{}^T \mathbf{x} + \frac{1}{2} \mathbf{y}^T Q \mathbf{y} + d_y{}^T \mathbf{y} + \text{const}$$

- Quadratic Placement Framework:

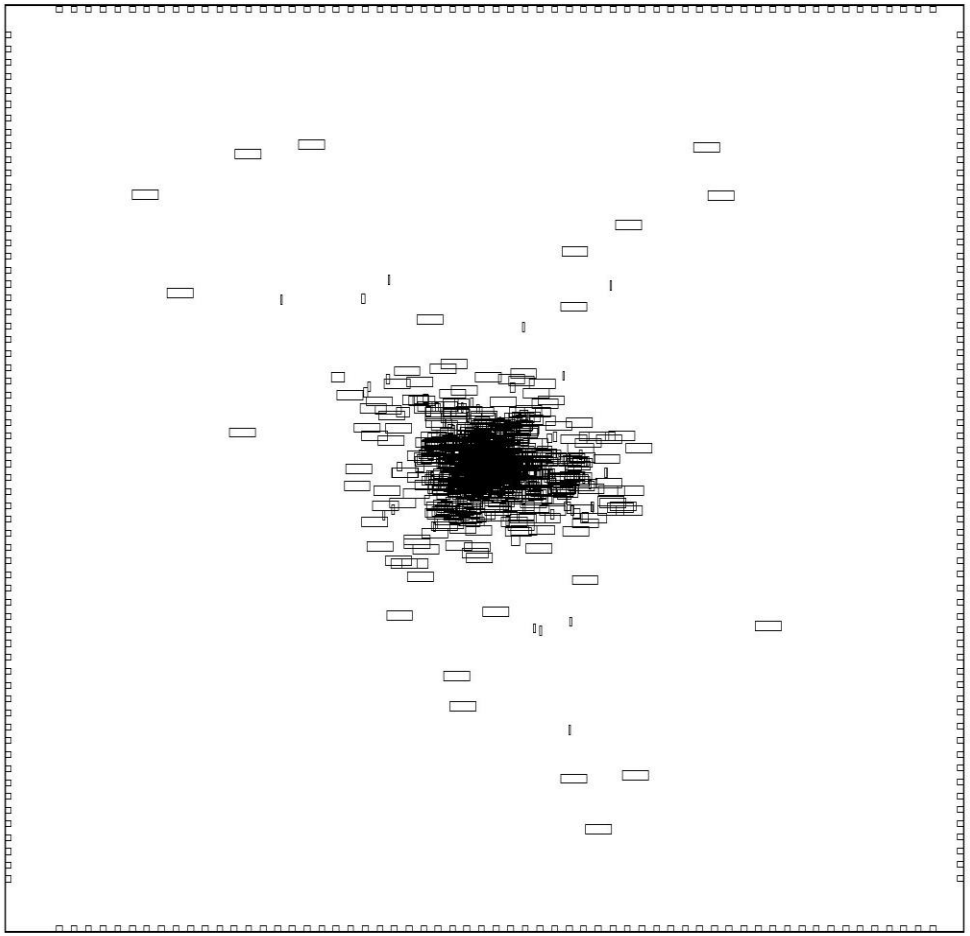
  repeat

  Solve the convex quadratic program

  Spread the cells

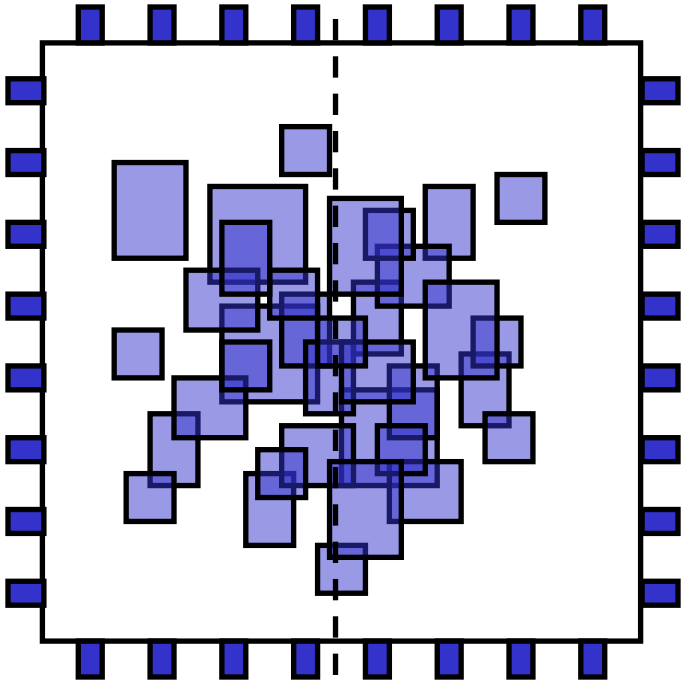
  until the cells are evenly distributed

Solution of the Original QP



Partitioning

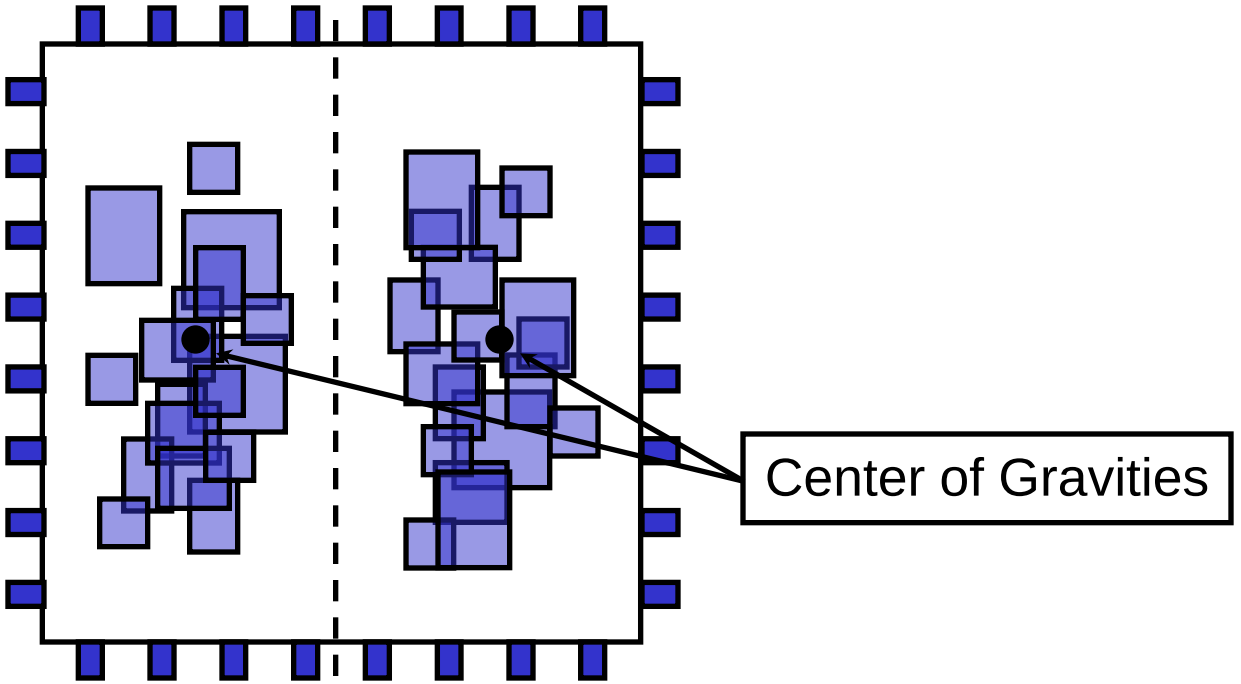
- Find a good cut direction and position



- Improve the cut value using FM

Applying the Idea Recursively

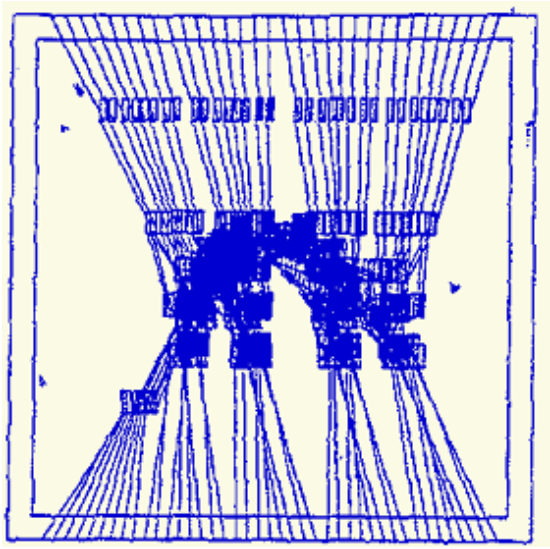
- Before every level of partitioning, do the Global Optimization again with additional constraints that the center of gravities should be in the center of regions



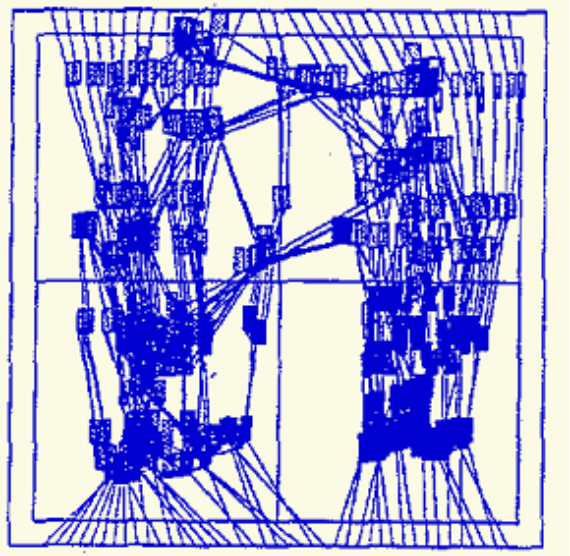
- Always solve a single QP (i.e., global)

Process of Gordian

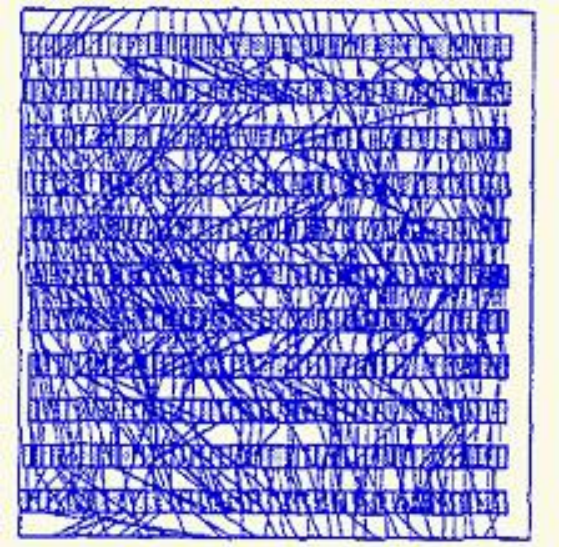
Global placement with 1 region



Global placement with 4 region



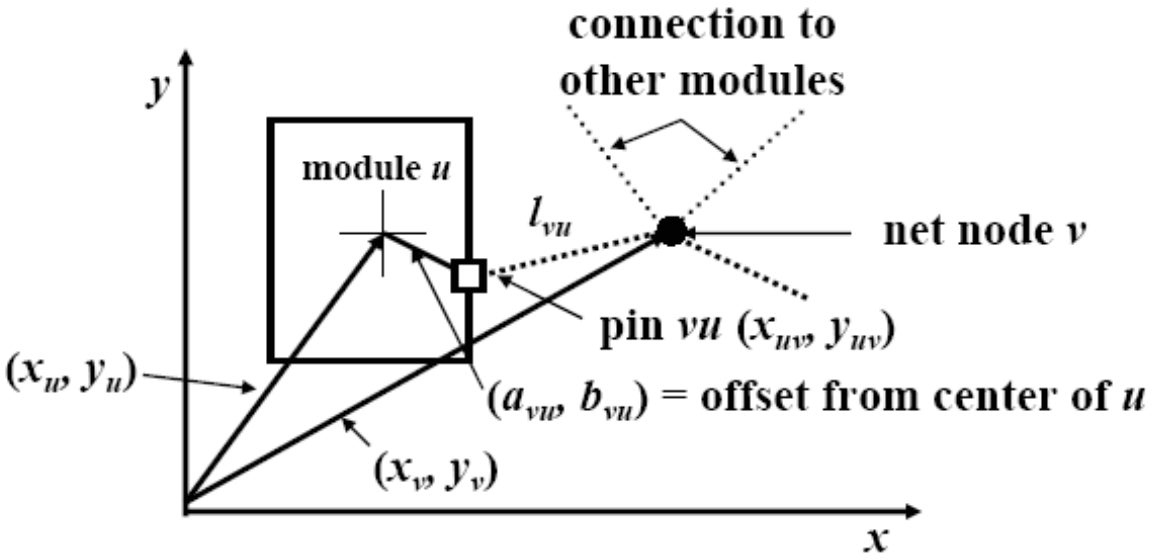
Final placements



Gordian Pseudo-code

```
Procedure Gordian
    l:=1;
    global-optimize(l);
    while (there exists |M_l|>k)
        for each r ϵ R(l)
            partition(r, r', r");
        l++;
        setup-constraints(l);
        global-optimize(l);
        repartition(l);
    final-placement(l);
endprocedure
```

Star Model for Wire Length



Squared wire length of net v

$$L_v = \sum_{u \in M_v} [(x_{uv} - x_v)^2 + (y_{uv} - y_v)^2]$$

$$x_{uv} = x_u + a_{vu},\ y_{uv} = y_u + b_{vu}$$

Center of Gravity Constraints

The center of gravity constraints

- At level l, chip is divided into q ($\leq 2^l$) regions
- For region p, the center coordinates: (u_p, v_p)
- M_p: set of modules in region p
- Matrix from for all regions

$$\text{constraint}: \sum_{u \in M_p} F_u x_u = u_p \sum_{u \in M_p} F_u$$

$$A^l X = u^l, a_{iu} = \begin{cases} F_i / \sum_{i \in M_p} F_i & \text{if } i \in M_p \\ 0 & \text{otherwise} \end{cases}$$

Solution Methods

- **Algebraic Manipulation**

$$A_{q\times m} = [D_{q\times q} B_{q\times(m-q)}]$$

$$[DB]\begin{bmatrix} X_d \\ X_i \end{bmatrix} = u, \text{ where } \begin{matrix} X_d \text{ is dependent variable} \\ X_i \text{ is independent variable} \end{matrix}$$

$$X_d = -D^{-1}BX_i + D^{-1}u$$

$$X = \begin{bmatrix} -D^{-1}B \\ I \end{bmatrix} X_i + \begin{bmatrix} D^{-1}u \\ 0 \end{bmatrix} = ZX + x_0$$

- **Unconstrained Quadratic Programming problem**
  - **Solved by Conjugate-Gradient method**

$$\text{UQP}: \min_{x_i \in R^{m-q}} \{\Psi(x_i) = X_i^T Z^T C Z X + C^T X_i\}, \text{ where } C^T = CX_0 + d$$

Other Details

- Sometimes, there are too much overlapping between modules (indicating a bad partitioning)
  - Merge 2 sub-partitions and repartition them is required
- When the size of sub-circuits are small enough, stop partitioning
  - Perform a final placement of the sub-circuits according to the design style

Repartitioning

- Module exchange after each cut to improve cut size
  - terminal propagation using global placement positions
- Repartitioning
  - to 'undo' the mistake made at the previous level:

```
Procedure repartition(l)
    if overlap exists
        for each r∈R(l-1)
            merge-regions(r, r', r'');
            partition(r, r', r'');
        setup-constraints(l);
        global-optimize(l);
    endif
```

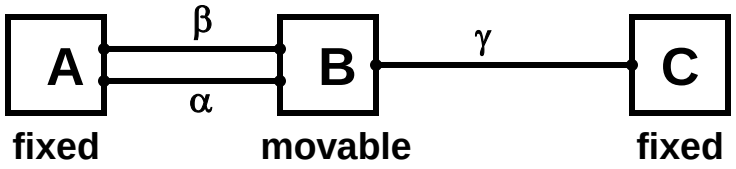
Quadratic Techniques

- Pros:
  - Mathematically well behaved
  - Efficient solution techniques find global optimum
  - Great quality

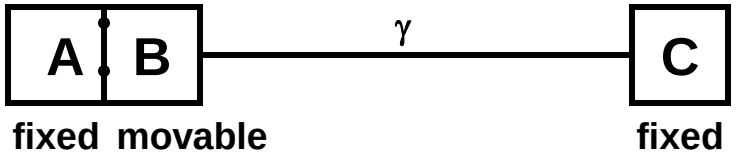
- Cons:
  - Solution of Ax + B = 0 is not a legal placement, so generally some additional partitioning techniques are required
  - Solution of Ax + B = 0 is that of the "mapped" problem, i.e., nets are represented as cliques, and the solution minimizes wire length squared, not linear wire length unless additional methods are deployed
  - Fixed IOs are required for these techniques to work well

Linear vs. Quadratic Objective Function

- Differences between linear and quadratic objective function



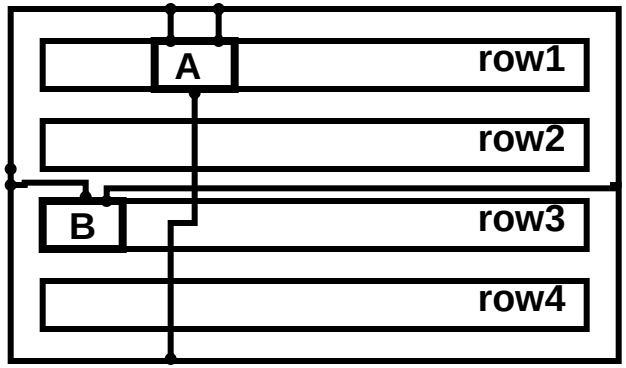
a) Quadratic objective function



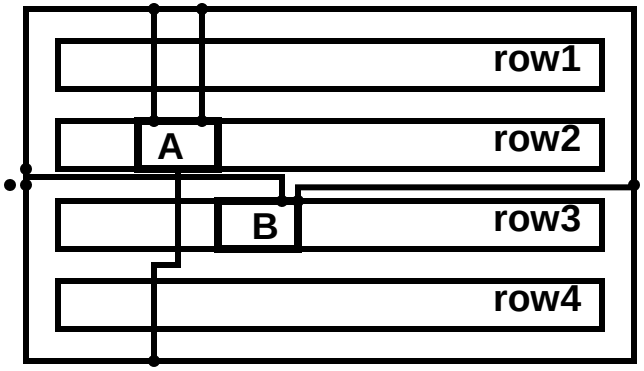
b) Linear objective function

Linear vs. Quadratic Objective Function

- Quadratic objective function tends to make very long net shorter than linear objective function does, and let short nets become slightly longer



Linear objective function

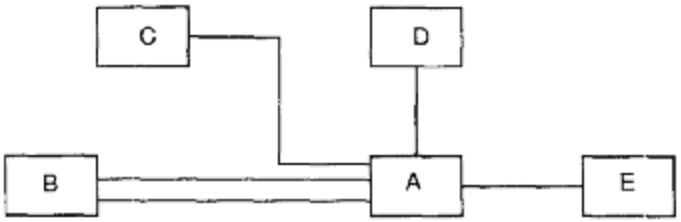


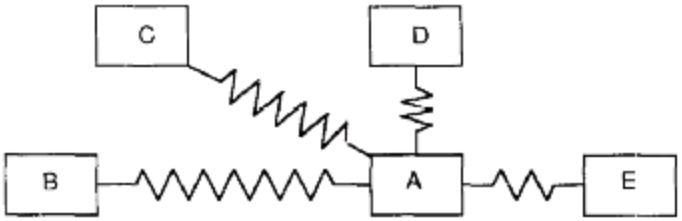
Quadratic objective function

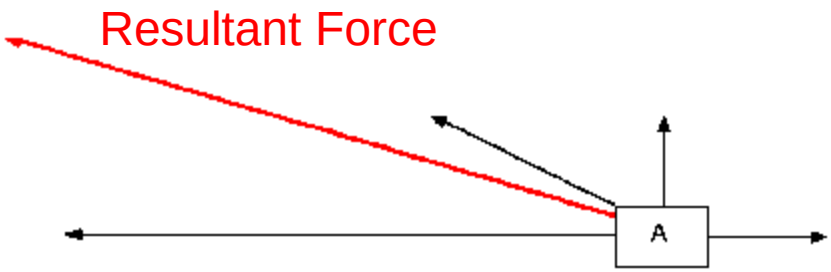
Force Directed Approach

- Transform the placement problem to the classical mechanics problem of a system of objects attached to springs
- Analogies:
  - Module (Block/Cell/Gate) = Object
  - Net = Spring
  - Net weight = Spring constant
  - Optimal placement = Equilibrium configuration

An Example







Force Calculation

- Hooke's Law:
  - Force = Spring Constant × Distance
- Can consider forces in x- and y-direction separately:

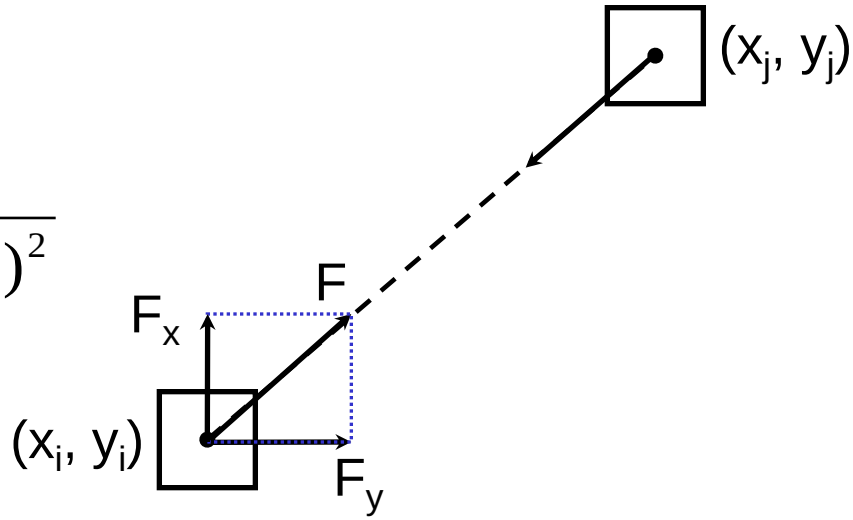
$$\text{Distance } d_{ij} = \sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$$

$$\text{Net Cost } c_{ij}$$

$$F = c_{ij}\sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}$$

$$F_x = c_{ij}(x_j - x_i)$$

$$F_y = c_{ij}(y_j - y_i)$$



Problem Formulation

- Equilibrium: $\Sigma_j c_{ij} (x_j - x_i) = 0$ for all module i.
- However, trivial solution: $x_j = x_i$ for all i, j. Everything placed on the same position!
- Need to have some way to avoid overlapping
- Several possible ways to avoid overlapping:
  - Add some repulsive force which is inversely proportional to distance (or distance squared)
  - Have connections to fixed I/O pins on the boundary of the placement region
  - Not really move to the equilibrium position, but a nearby position without introducing overlapping

Algorithms for Force-Directed Placement

- Many variations exist, 2 major categories below.

- Constructive Methods:
  - Coordinates of all modules are treated as variables
  - Solving all equations simultaneously

- Iterative Methods:
  - Start with some initial placement, improve placement by moving modules iteratively
  - Select one module at a time
  - Move that module to the point of zero force

    (assume positions of all other modules are fixed)

Comments on Force-Directed Placement

- ✓ Use directions of forces to guide the search (compared with 'blind' search of simulated annealing)
- ✓ Usually much faster than simulated annealing
- x Focus on connections, not shapes of blocks
- x Only a heuristic; an equilibrium configuration does not necessarily give a good placement
- ? Successful or not depends on the way to eliminate overlapping

Kraftwerk Placement Tool

Hans Eisenmann and F. Johannes,
“Generic Global Placement and Floorplanning”,
DAC-98, p.269 - 274

Approach

- Iteratively solve the quadratic formulation:

$$\text{Min} \quad f(p) = \tfrac{1}{2} p^T C p + d^T p + const$$

$$\Rightarrow Cp + d = 0 \qquad \textbf{// equivalent to spring force equilibrium}$$

- Spread cells by additional forces:

$$C p + d + f = 0$$

- Density-based force proposed
  - Push cells away from dense region to sparse region

$$f(x,y) = \frac{k}{2\pi} \int_{-\infty}^{\infty} \int_{-\infty}^{\infty} D(x',y') \frac{\vec{r} - \vec{r}'}{\left|\vec{r} - \vec{r}'\right|^2} dx'dy'$$

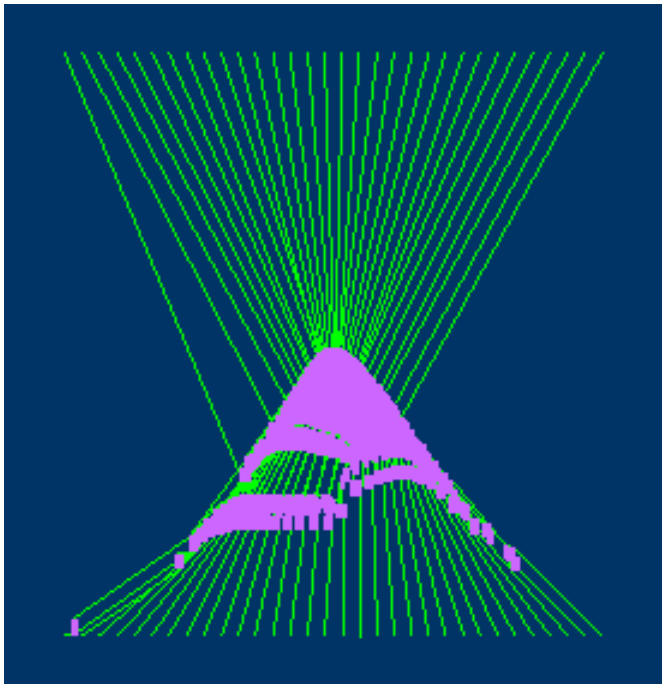
where $\vec{r} = (x, y)$ and $\vec{r}' = (x', y')$

Some Details

- Let f_i be the additional force applied to cell i
- The proportional constant k is chosen so that the maximum of all f_i is the same as the force of a net with length $K(W+H)$
- K is a user-defined parameter
  - K=0.2 for standard operation
  - K=1.0 for fast operation
- Can be extended to handle timing, mixed block placement and floorplanning, congestion, heat-driven placement, incremental changes, etc.

Some Potential Problems of Kraftwerk

- Convergence is difficult to control
  - Large K → oscillation
  - Small K → slow convergence
  - Example:
    - Layout of a multiplier



- Density-based force is expensive to compute

$$f(x,y) = \frac{k}{2\pi}\int_{-\infty}^{\infty}\int_{-\infty}^{\infty} D(x',y')\frac{\vec{r}-\vec{r}'}{\left|\vec{r}-\vec{r}'\right|^2}dx'dy'$$

FastPlace: Efficient Analytical Placement using Cell Shifting, Iterative Local Refinement and a Hybrid Net Model

Natarajan Viswanathan and Chris Chu

ISPD-04

FastPlace Approach

- FastPlace Framework (roughly):

  repeat

  Solve the convex quadratic program
  Reduce wire length by iterative heuristic
  Spread the cells

  until the cells are evenly distributed

- Special features of FastPlace:
  - Hybrid Net Model
    - Speed up solving of convex QP
  - Cell Shifting
    - Easy-to-compute technique
    - Enable fast convergence
  - Iterative Local Refinement
    - Minimize wire length based on linear objective

Effect of Net Model on Runtime

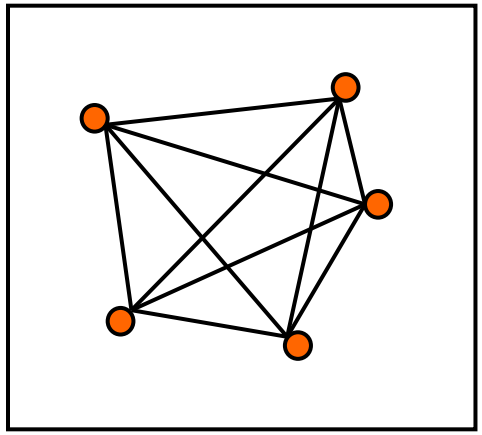
- Need to replace each mulit-pin net by 2-pin nets
- Then the placement problem (even with pseudo nets) can be formulated as a convex QP:

$$\text{Total cost} = \frac{1}{2}\mathbf{x}^T Q\mathbf{x} + d_x{}^T\mathbf{x} + \frac{1}{2}\mathbf{y}^T Q\mathbf{y} + d_y{}^T\mathbf{y} + \text{const}$$

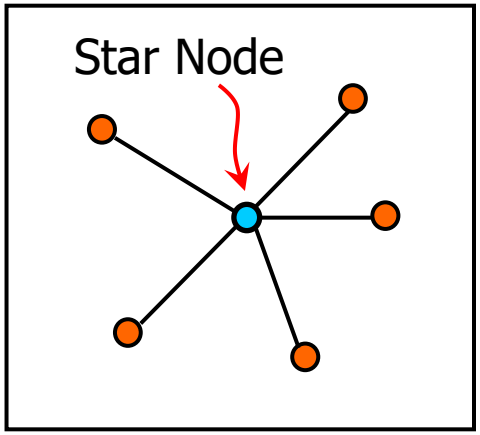
- Solved by any convex QP algorithms
  - Use Incomplete Cholesky Conjugate Gradient (ICCG)
- Runtime is proportional to # of non-zero entries in Q
- Each non-zero entry in Q corresponds to one 2-pin net
- Traditionally, placers model each multi-pin net by a clique
- High-degree nets will generate a lot of 2-pin nets
- Slow down convex QP algorithms significantly

Clique, Star and Hybrid Net Models

- Star model is introduced by Mo et al. [ICCAD-00] for macro placement
- Introduce a star node even for 2-pin nets
- Not clear how the placement result will be affected



Clique Model

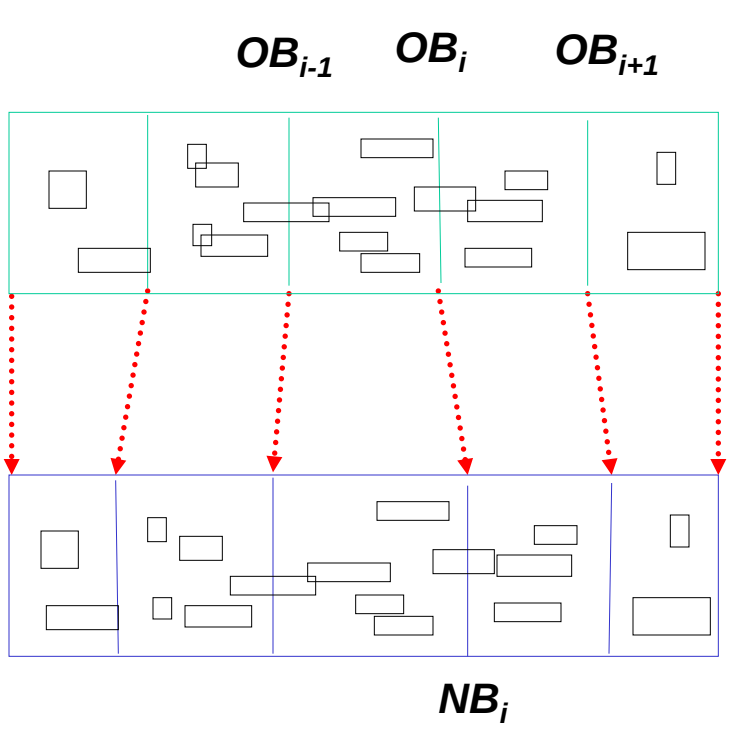


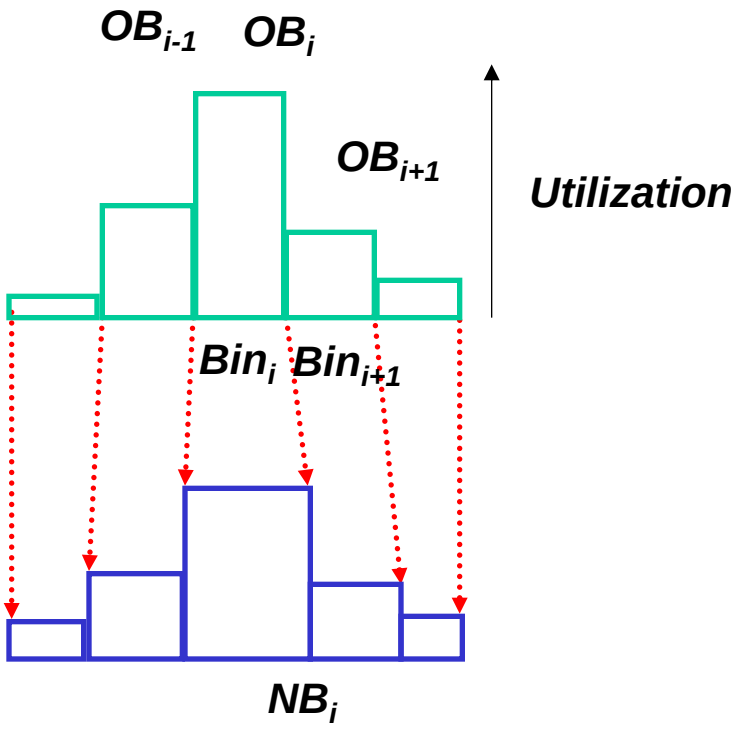
Star Model

# pins	Net Model
2	Clique
3	Clique
4	Star
5	Star
…	…

Hybrid Model

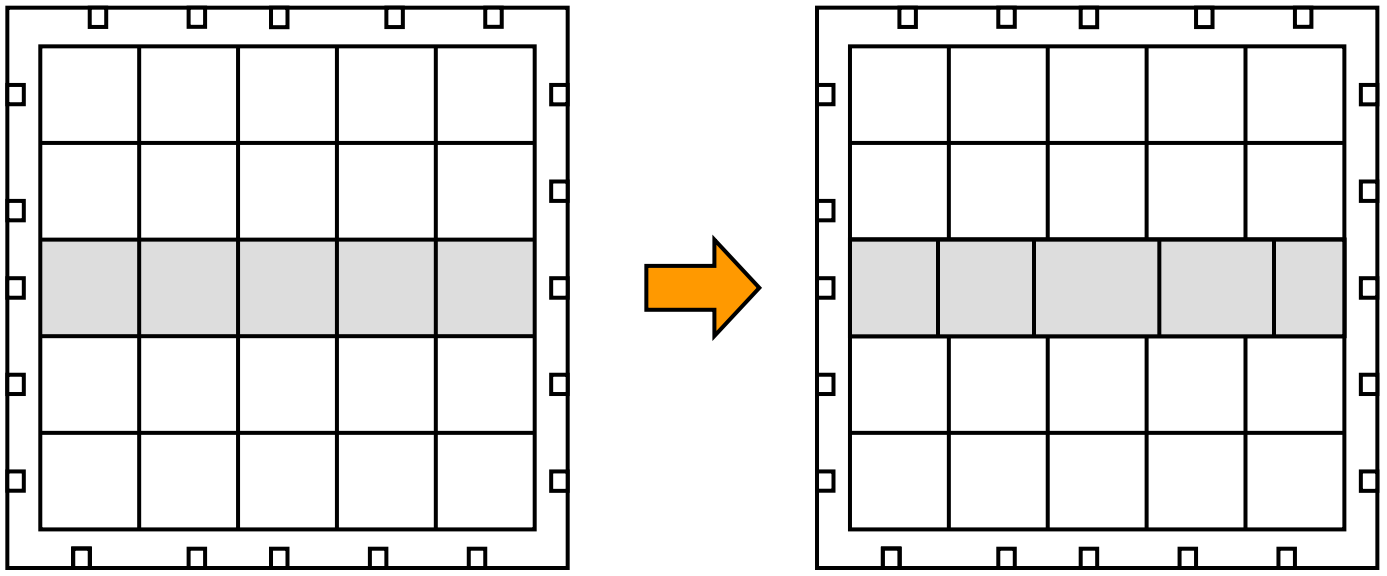
Cell Shifting





Cell Shifting

- Shifting of bin boundary



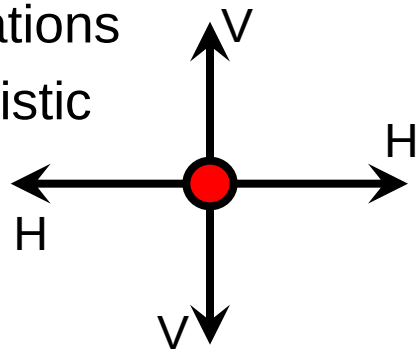
Uniform Bin Structure

Non-uniform Bin Structure

- Shifting of cells linearly within each bin
  - Apply to all rows and all columns independently

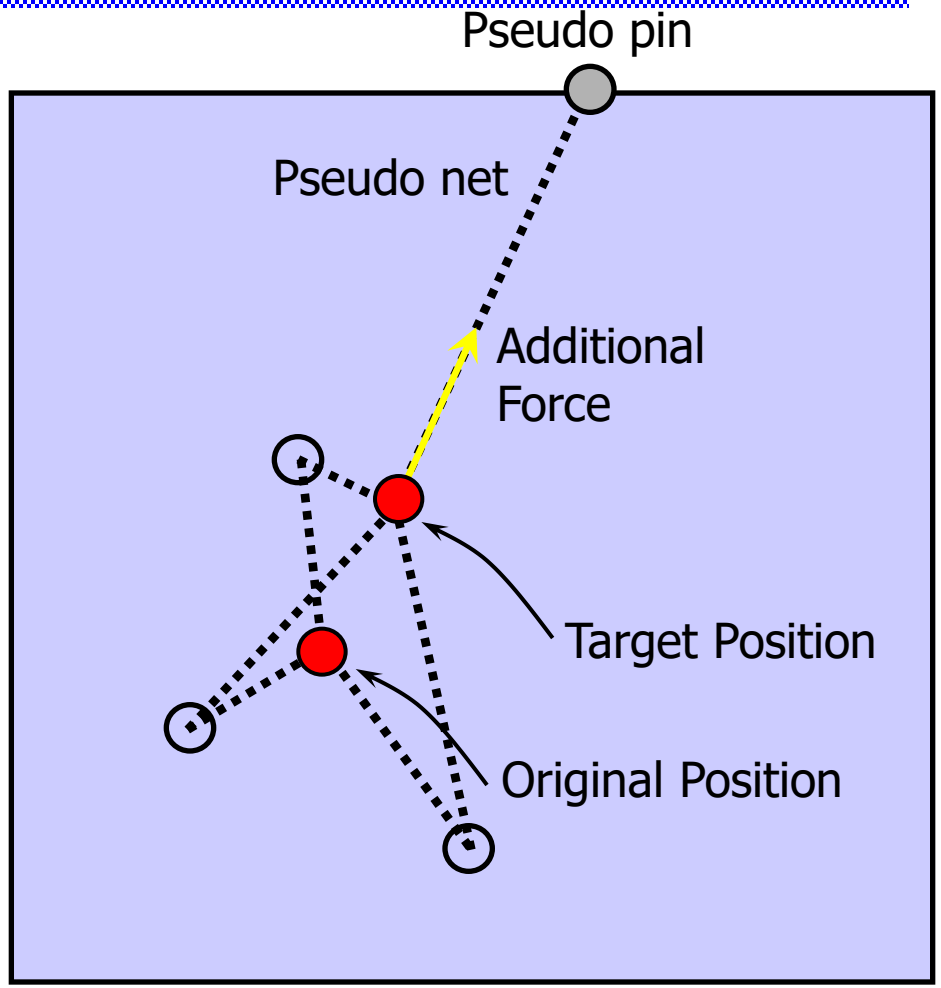
Iterative Local Refinement

- Iteratively go through all the cells one by one
- For each cell, consider moving it in four directions for a certain distance
- Compute a score for each direction based on
  - Half-perimeter wire length (HPWL) reduction
  - Cell density at the source and destination regions
- Move to the direction with highest positive score (No move if no positive score)
- Distance moved (H or V) is decreasing over iterations
- Detailed placement is handled by the same heuristic



Pseudo Pin and Pseudo Net

- Need to add extra forces to prevent cells from collapse back
- Extra forces are introduced by adding pseudo pins and pseudo nets
- Only diagonal and linear terms of the quadratic system need to be updated
- Horizontal and vertical problems have the same connectivity matrix Q



Comparison

- FastPlace is fast:
  - Compare to Capo 8.8:
    - 13x faster
    - 1% longer in wire length
  - Compare to Dragon 2.2.3
    - 97.4x faster
    - 1.6% longer in wire length
  - Compare to Kraftwerk (based on published data)
    - 20-25x faster
    - 10% better in wire length
- Lots of small improvement and implementation tricks
  - Still need to understand how and why it works

Summary

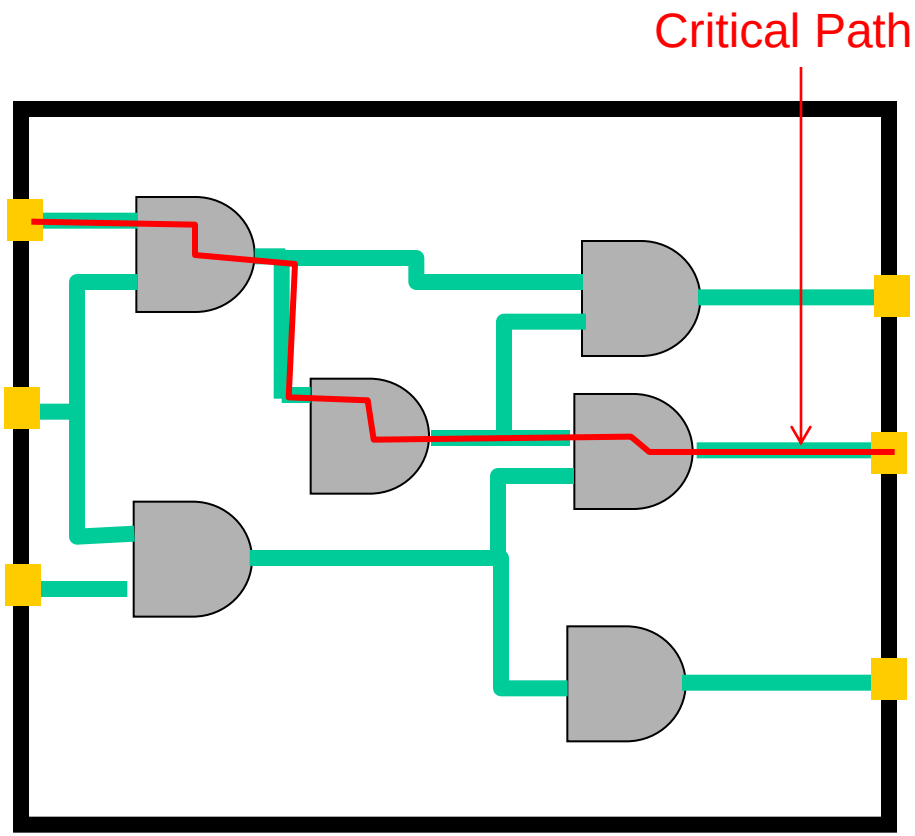
- We have covered three main methods of placement
  - Simulated Annealing
  - Partition-based methods
  - Analytical methods
- Each of them has pros and cons
- In practice, hybrid (of these techniques) is used
  - Quadratic with partitioning
  - Simulated annealing with clustering (dual of partition)

Outline

- Wire length driven placement
- Main methods
  - Simulated Annealing
  - Partition-based methods
  - Analytical methods
  - Timing, congestion and other considerations
- Global placement (rough location)
- Detailed placement (legalization)

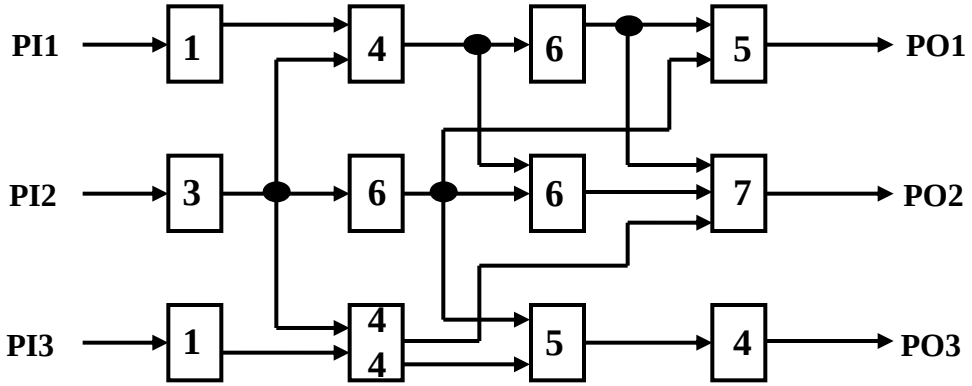
Timing Cost

- Delay of the circuit is defined as the longest delay among all possible paths from primary inputs to primary outputs
- Interconnection delay becomes more and more important in deep sub-micron regime

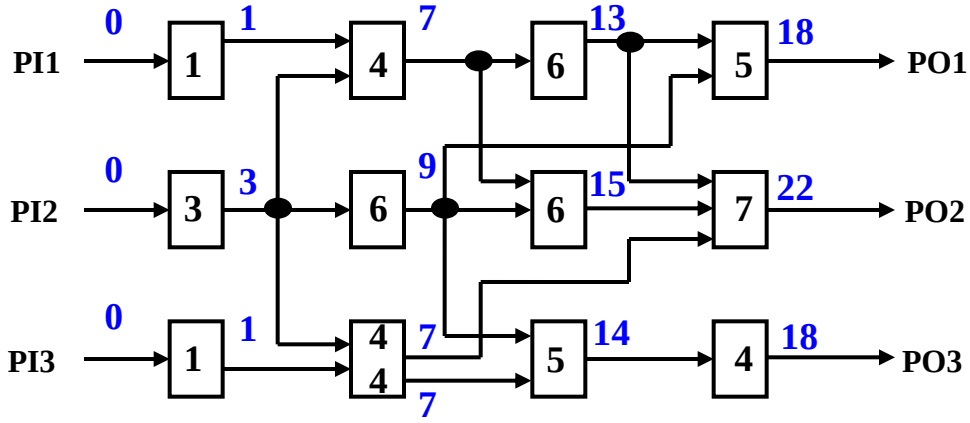


Timing Analysis (1/3)

Netlist with gate delay

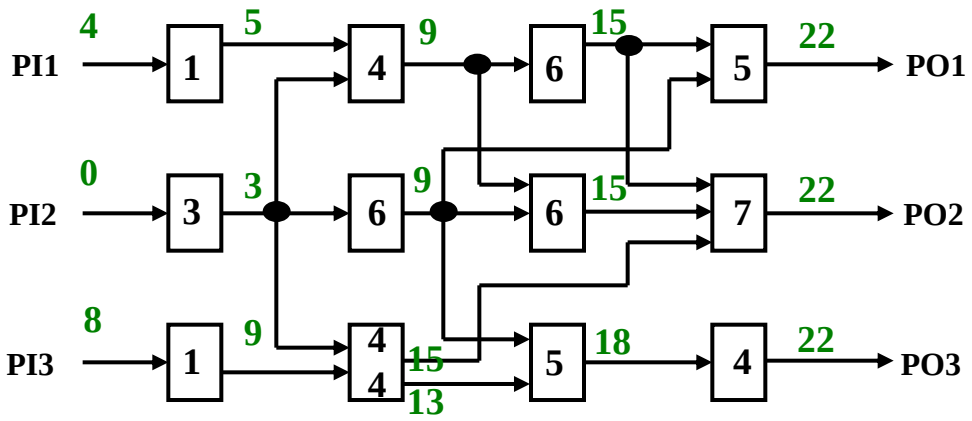


Arrival time (ASAP)

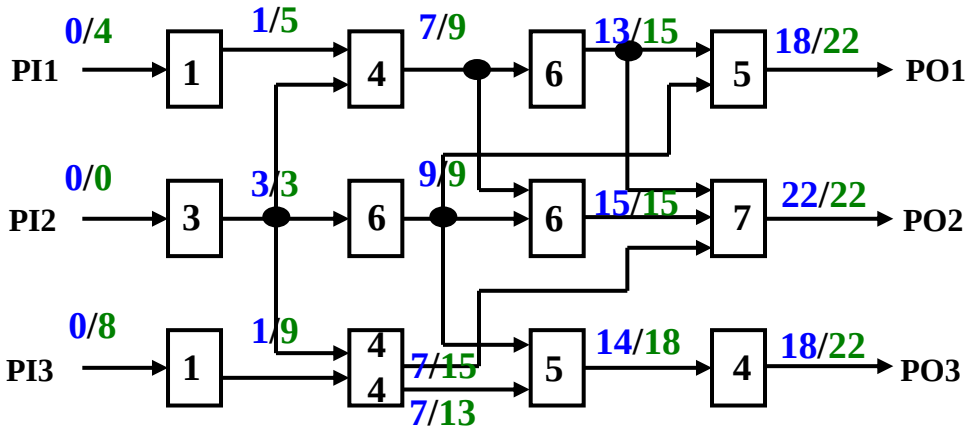


Timing Analysis (2/3)

Required time (ALAP)

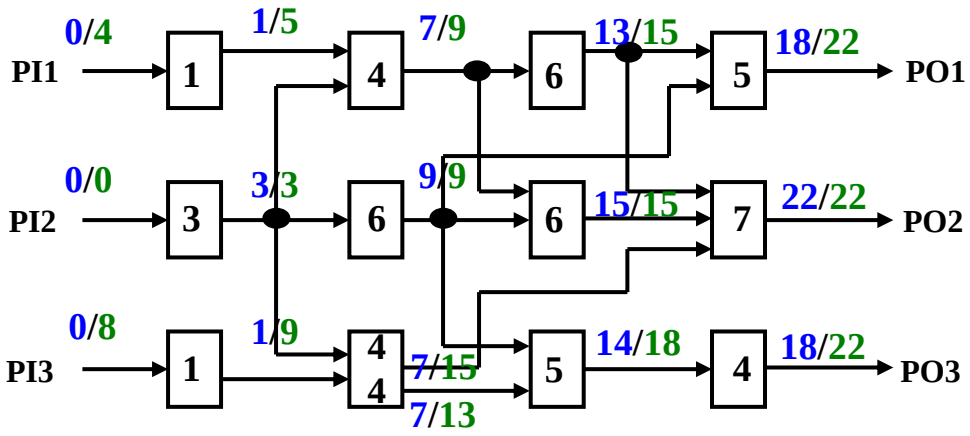


Arrival time/Required time

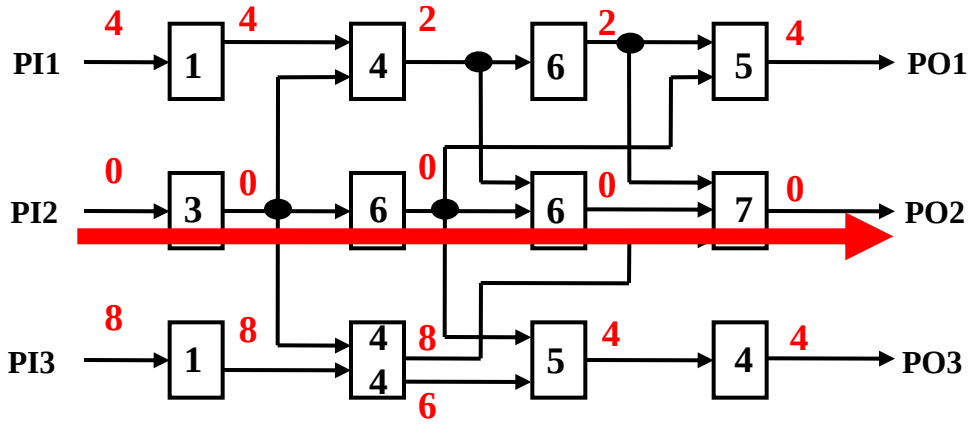


Timing Analysis (3/3)

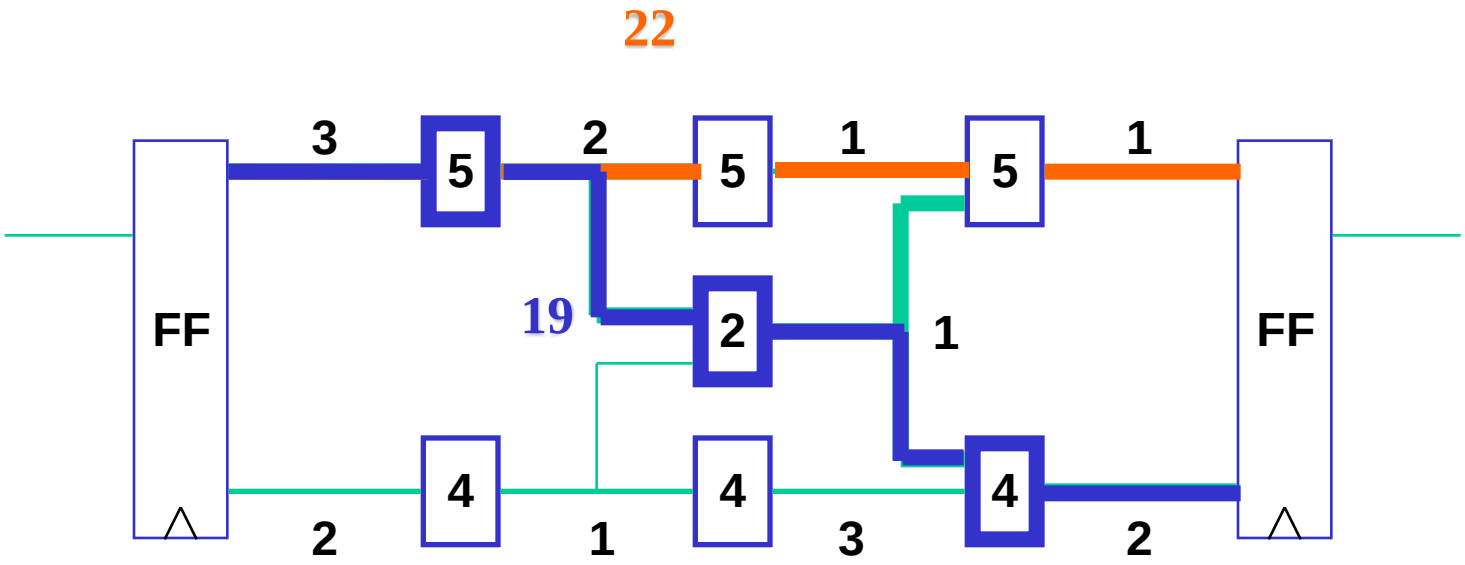
Arrival time/Required time



Slack (餘裕) =
Required time – Arrival time



Another example with interconnect delay – Same Timing Analysis



Timing Driven Placement Approaches

- Path-based
  - Most accurate information
  - Very slow
- Budgeting
  - Inaccurate information
  - Hard to budget
  - Fast
- Net-based approach
  - Net-weighting

Net-Weighting

- Basic approach
  - For more timing critical nets (i.e., smaller slack), assign higher net weights
  - Minimize

$$\sum_i w_i \cdot net_length(i)$$

  - where

$$w_i \propto \frac{1}{S_i}$$

Sensitivity Guided Netweighting for Placement Driven Synthesis

H. Ren, D. Z. Pan and D.S. Kung

ISPD-04

Figure of Merit (FOM)

- FOM is the total slack difference compared to a certain slack threshold for all timing end points

$$FOM = \sum_{Slk(t)<Slk_t}^{t \in P_o} (Slk(t) - Slk_t)$$

- Interpreted as the amount of work left for the physical synthesis engine or to the designers for manual fix
- FOM and WNS (worst negative slack) are the two most important metrics for timing closure in modern physical synthesis
- However, FOM was not used to guide placement explicitly

Sensitivity Definitions

- Net length sensitivity to net weight:

$$S_W^L = \frac{\Delta L}{\Delta W}$$

- Net delay sensitivity to net length:

$$S_L^T = \frac{\Delta T}{\Delta L}$$

- Net slack sensitivity to net weight:

$$S_W^{Slk} = -\frac{\Delta T}{\Delta W} = -\frac{\Delta T}{\Delta L} \cdot \frac{\Delta L}{\Delta W} \equiv -S_L^T \cdot S_W^L \equiv -S_W^T$$

- FOM sensitivity to net delay:

$$S_T^{FOM} = \frac{\Delta FOM}{\Delta T}$$

- FOM sensitivity to net weight:

$$S_W^{FOM} = \frac{\Delta FOM}{\Delta W} = \frac{\Delta FOM}{\Delta T} \cdot \frac{\Delta T}{\Delta W} \equiv S_T^{FOM} \cdot S_W^T$$

Closed-Form Sensitivity

- For net length to weight sensitivity, we have

$$S_W^L = -L \cdot \frac{W_{src} + W_{sink} - 2W}{W_{src} W_{sink}}$$

- For delay to wire length sensitivity, we have

$$S_L^T = \frac{\Delta T}{\Delta L} = rcL + cR_d + rC_l$$

- Use switch-level RC and Elmore delay to illustrate the concept
  - Good enough during placement
  - Can be extended to more accurate models

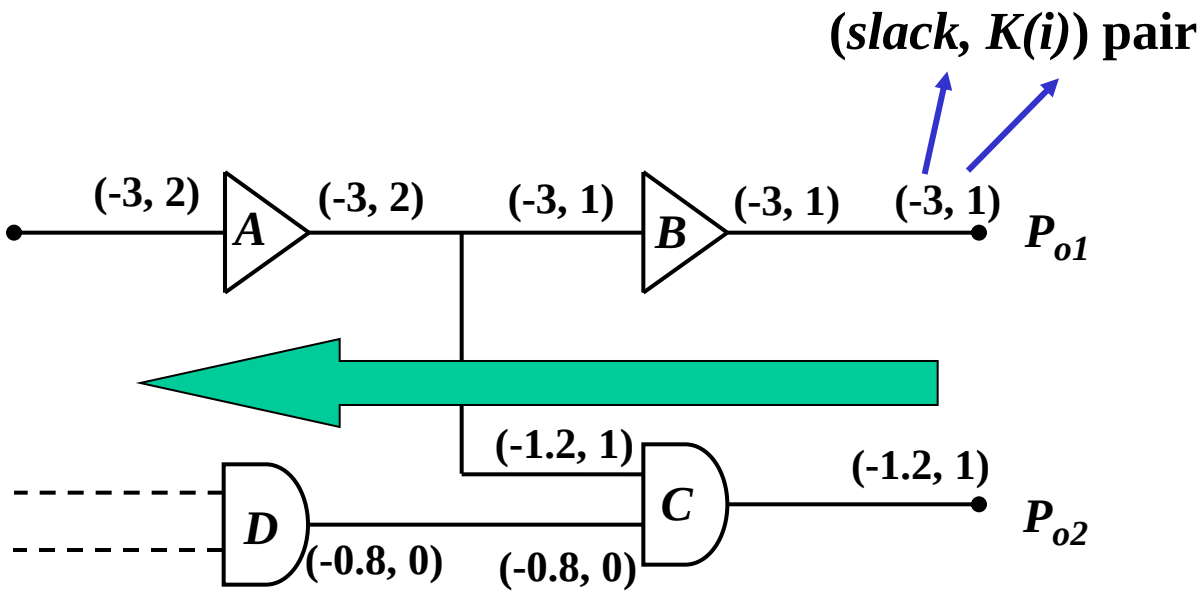
FOM to Net Delay Sensitivity

- Question: suppose the delay of net *i* is reduced by a small amount $\Delta T(i)$, what is the impact to FOM?
- Define: $K(i)$ to be the number of timing end points whose slack will change due to $\Delta T(i)$
- Then, we have the following theorem

$$S_T^{FOM}(i) \equiv \frac{\Delta FOM}{\Delta T(i)} = -K(i)$$

K(i) Computation

- Topologically sorted order from PO to PI
- Only propagate *K(i)* to the most timing critical input pin



Net Weight Generation

- Put these sensitivities together and generate new net weight

$$\Delta W(i) = \beta\left[(Slk_t - Slk(i))S_w^{Slk}(i) + \alpha S_W^{FOM}(i)\right]$$

$$W(i) = \begin{cases} W_{org}(i) & Slk(i) > Slk_t \\ W_{org}(i) + \Delta W(i) & Slk(i) \le Slk_t \end{cases}$$

Experiments

- We compare the placement and physical synthesis results of three different algorithms on 7 industry chips (up to 444k movable objects) from IBM
  - WL: wire length driven placement with uniform weight
  - TS: timing driven placement using slack sensitivity
  - TSF: timing driven placement using both slack and FOM sensitivity

Timing after Placement

	FOM				Improvement	
Design	**ZW**	**WL**	**TS**	**TSF**	**TS**	**TSF**
ckt1	-9134	-41650	-26093	-25602	48%	49%
ckt2	0	-6966	-4102	-3454	41%	50%
ckt3	-535	-13711	-6468	-5595	55%	62%
ckt4	-322	-8057	-4024	-3440	52%	60%
ckt5	-114	-28527	-15334	-12229	46%	57%
ckt6	-142	-20257	-9417	-9536	54%	53%
ckt7	-4	-452	-248	-131	46%	72%
Average					49%	58%

	WNS				Improvement	
Design	**ZW**	**WL**	**TS**	**TSF**	**TS**	**TSF**
ckt1	-1.702	-6.274	-3.392	-4.254	63%	44%
ckt2	0.248	-2.977	-1.784	-1.754	37%	38%
ckt3	-0.55	-4.997	-3.684	-3.788	30%	27%
ckt4	-0.941	-7.218	-3.736	-3.605	55%	58%
ckt5	-0.102	-3.575	-2.379	-2.002	34%	45%
ckt6	-0.508	-5.47	-5.484	-4.856	0%	12%
ckt7	0.16	-1.135	-0.66	-0.432	37%	54%
Average					37%	40%

Timing after Physical Synthesis

	FOM			Improvement	
Design	**WL**	**TS**	**TSF**	**TS**	**TSF**
ckt1	-7829	-6086	-5170	22%	34%
ckt2	-2059	-384	-631	81%	69%
ckt3	-1854	-405	-422	78%	77%
ckt4	-2537	-1844	-1770	27%	30%
ckt5	-4732	-2726	-1819	42%	62%
ckt6	-1481	-541	-266	63%	82%
ckt7	-94	-8	0	91%	100%
Average				58%	65%

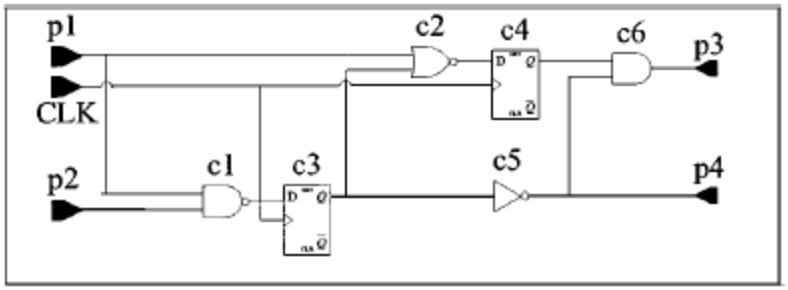
	WNS			Improvement	
Design	**WL**	**TS**	**TSF**	**TS**	**TSF**
ckt1	-0.834	-0.743	-0.739	11%	11%
ckt2	-0.705	-0.011	-0.073	98%	90%
ckt3	-0.701	-0.139	-0.19	80%	73%
ckt4	-2.156	-1.908	-1.9	12%	12%
ckt5	-0.472	-0.443	-0.341	6%	28%
ckt6	-0.36	-0.293	-0.351	19%	3%
ckt7	-0.097	0.182	0.283	100%	100%
Average				47%	45%

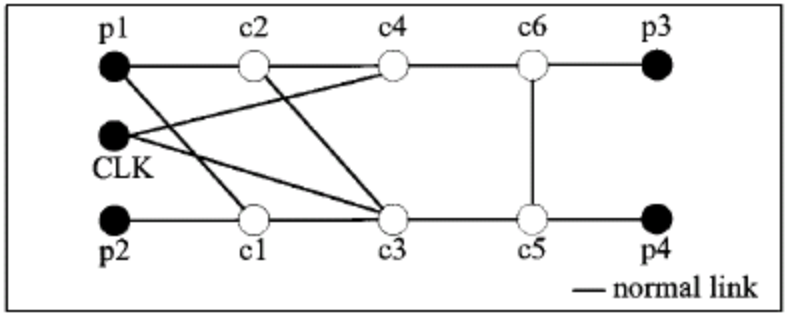
Effective Enforcement of Path-Delay Constraints in Performance-driven Placement

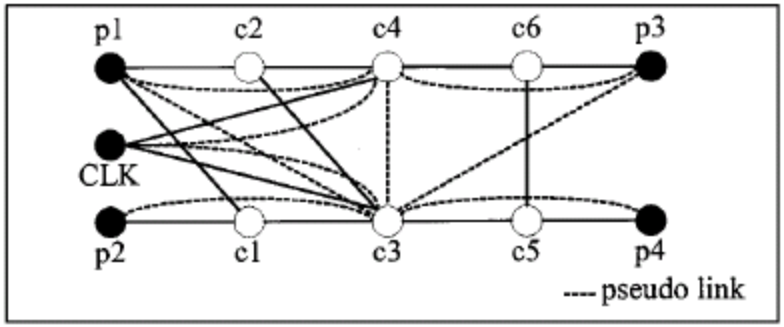
Y. C. Chou and Y. L. Lin

TCAD-02

Path-Delay Constraints



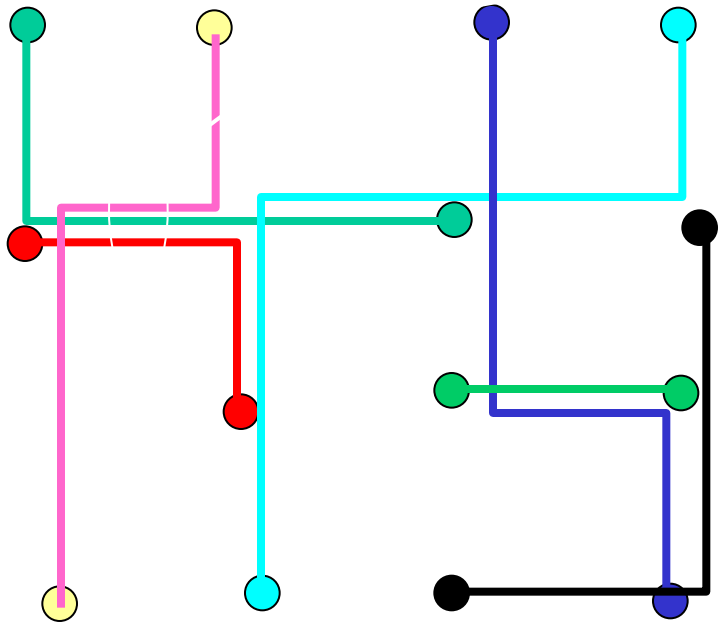




Congestion Minimization

- Traditional placement problem is to minimize interconnection length (wirelength)
- A valid placement has to be routable
- Congestion is important because it represents routability (lower congestion implies better routability)
- There is not yet enough research work on the congestion minimization problem

Definition of Congestion



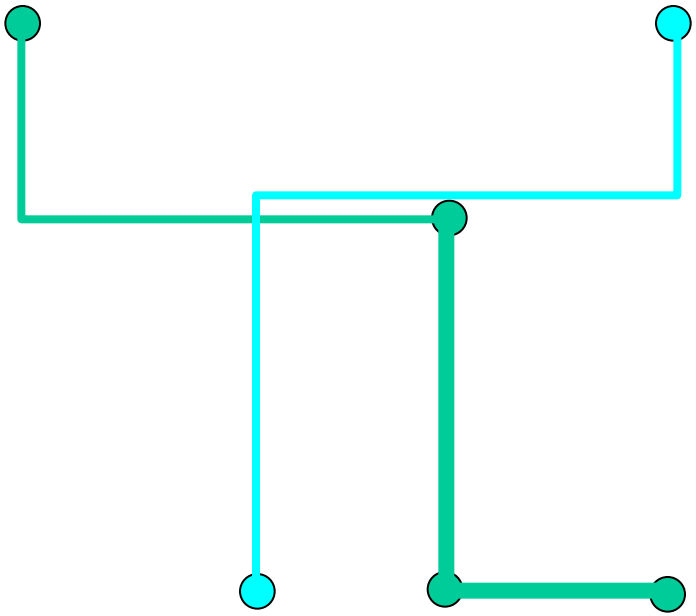
Routing demand = 3
Assume routing supply is 1,
overflow = 3 - 1 = 2 on this edge.

Overflow on each edge =

$$\begin{cases} \text{Routing Demand - Routing Supply} \\ \quad \text{(if Routing Demand > Routing Supply)} \\ 0 \quad \text{(otherwise)} \end{cases}$$

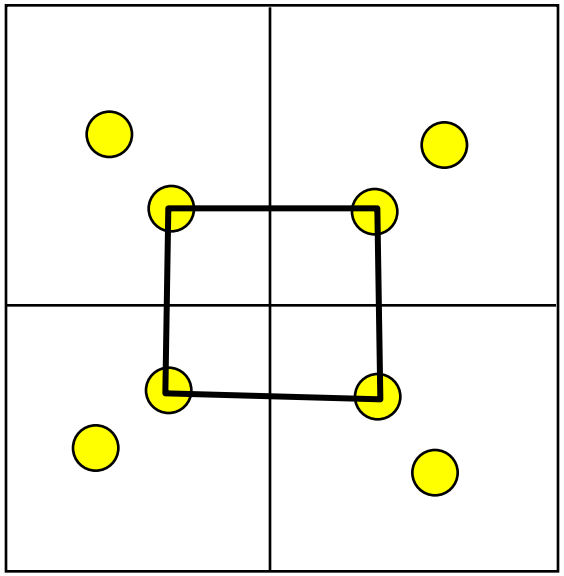
$$\text{Overflow} = \sum_{\text{all edges}} \text{overflow}$$

Correlation between Wirelength and Congestion

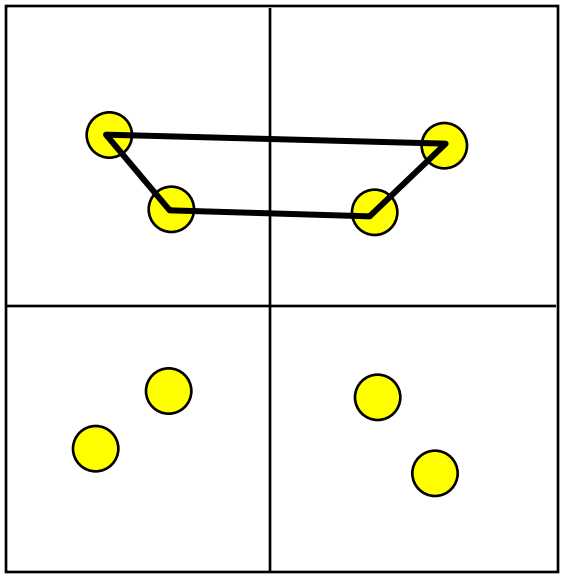


Total Wirelength = Total Routing Demand

Wirelength ≠ Congestion

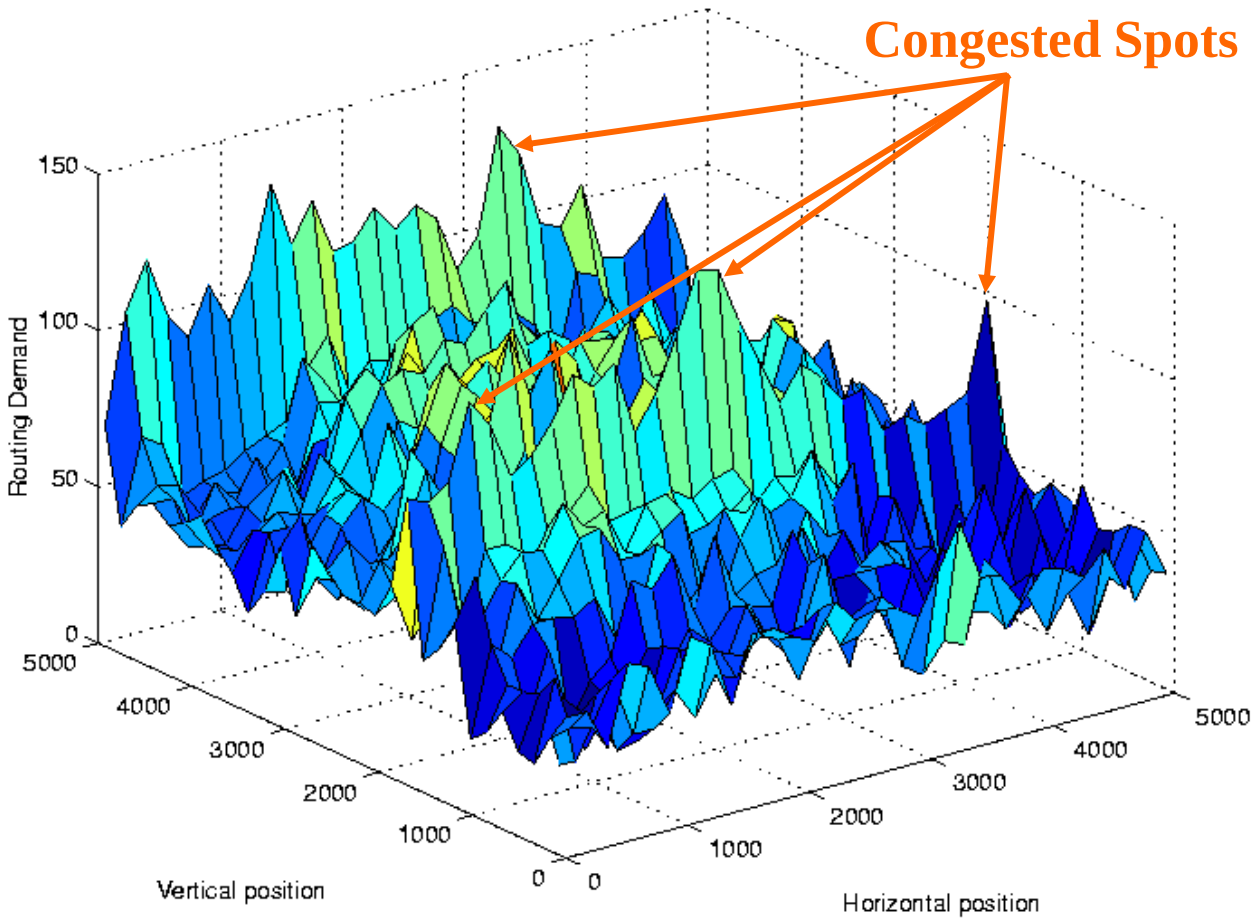


A congestion minimized placement



A wirelength minimized placement

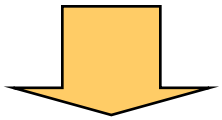
Congestion Map of a Wirelength Minimized Placement



Congestion Reduction Postprocessing



Reduce congestion globally by minimizing the traditional wirelength



Post process the wirelength optimized placement using the congestion objective



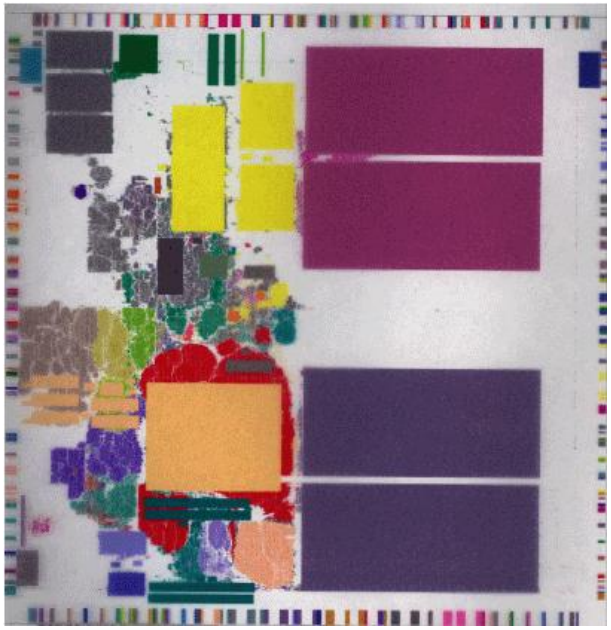
An Effective Congestion Driven Placement Framework

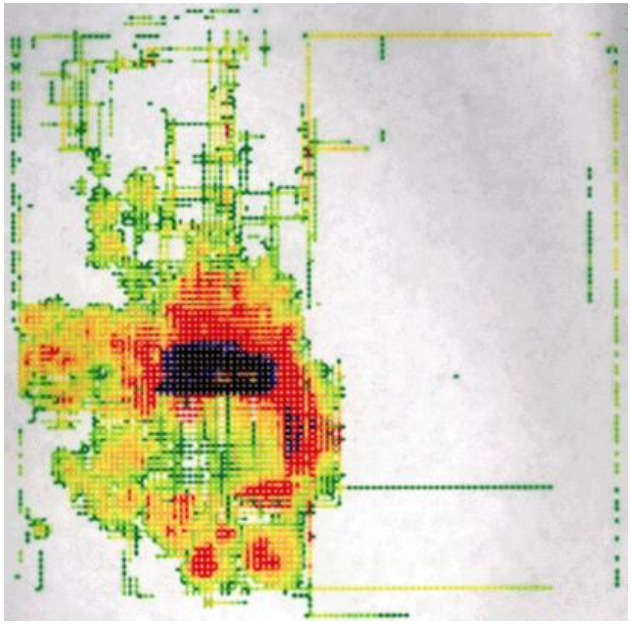
André Rohe
University of Bonn, Germany

joint work with Ulrich Brenner

ISPD 2002 (Best Paper)

A dense Placement





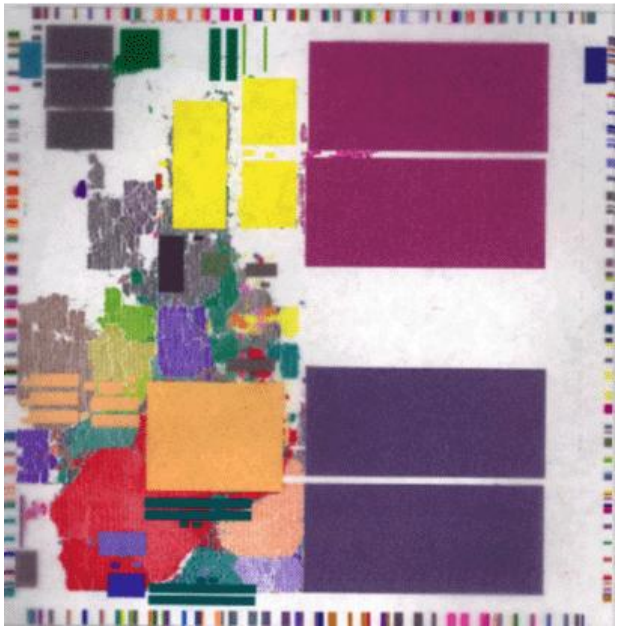
- Good wirelength, but…
- Impossible to route

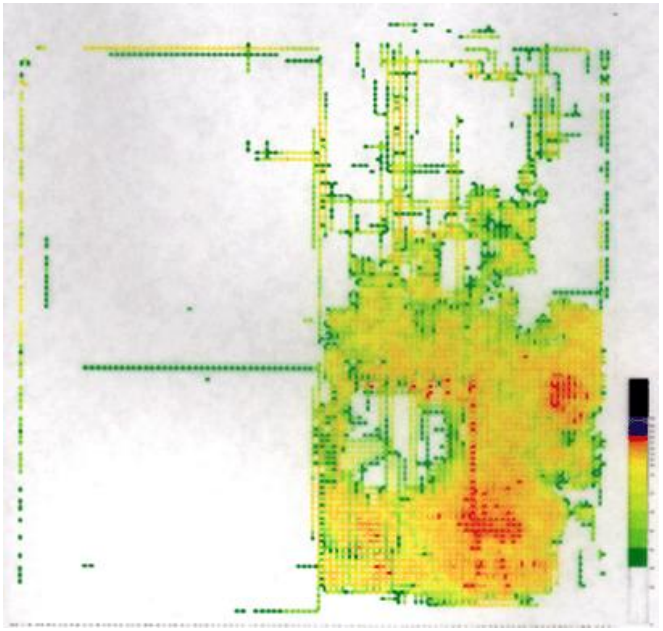
Possible Solution



- Easy to route, but…
- Bad wirelength/timing

Congestion Driven Placement





- Easy to route + good wirelength
- Almost no extra computation effort!

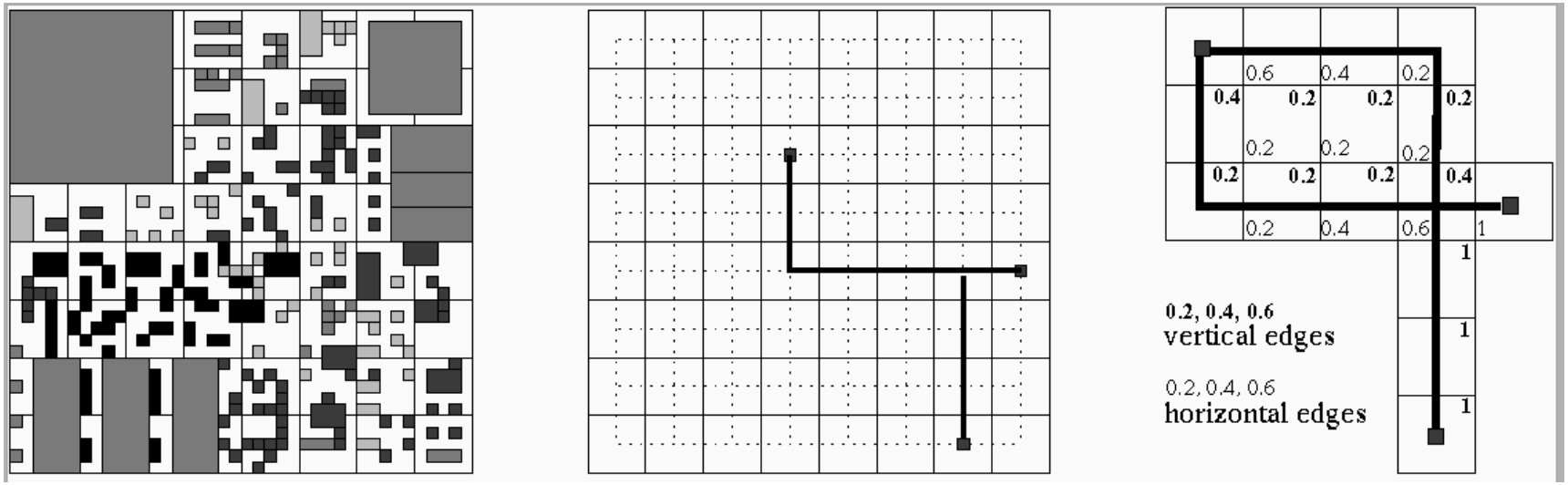
Overall Algorithm: Bonn Place

- Partitioning based approach
- Solves QP in each level, followed by partitioning
- Partitioning is done by quadrisection:
  circuits are partitioned with minimum movement
  (Vygen)

Methods used for congestion driven placement

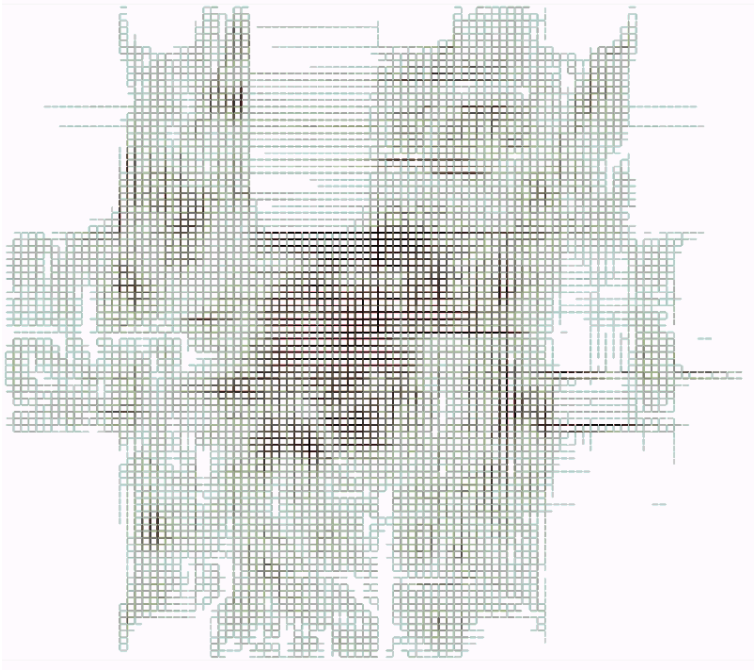
- Very fast congestion calculation
- Inflate circuits in congested regions
- Spreading inflated cells

Congestion Calculation



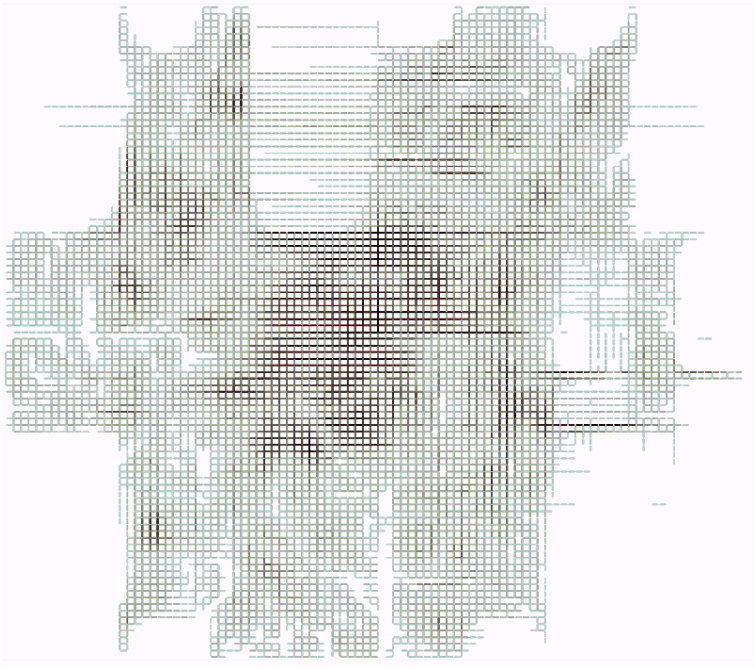
- Calculate Steiner Tree for each net
- Probablitiy estimation for each 2-point connection (similar to Hung & Flynn, Lou et al.)

Quality of Congestion Calculation

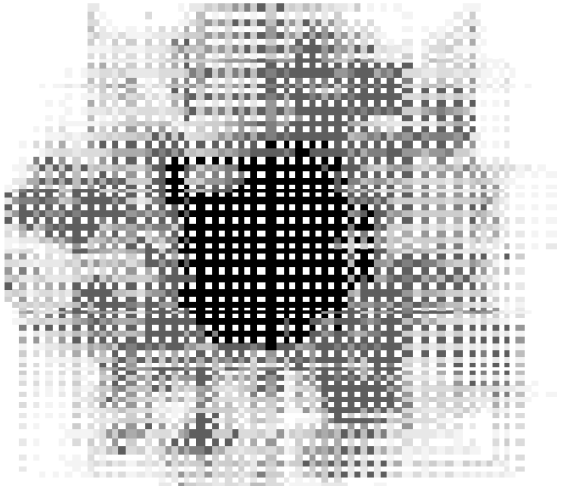


congestion estimation

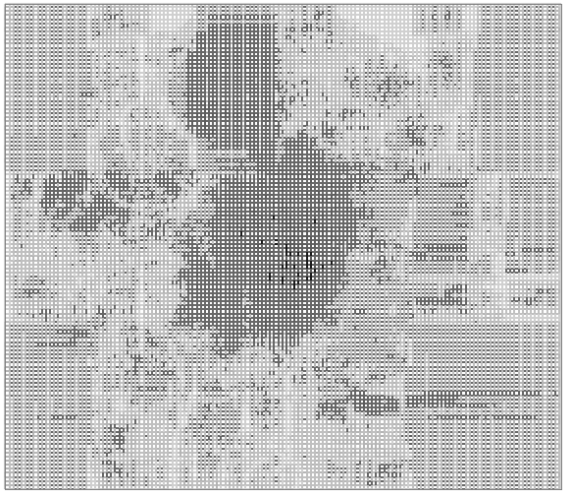
Quality of Congestion Calculation



congestion estimation



Bonn
Global

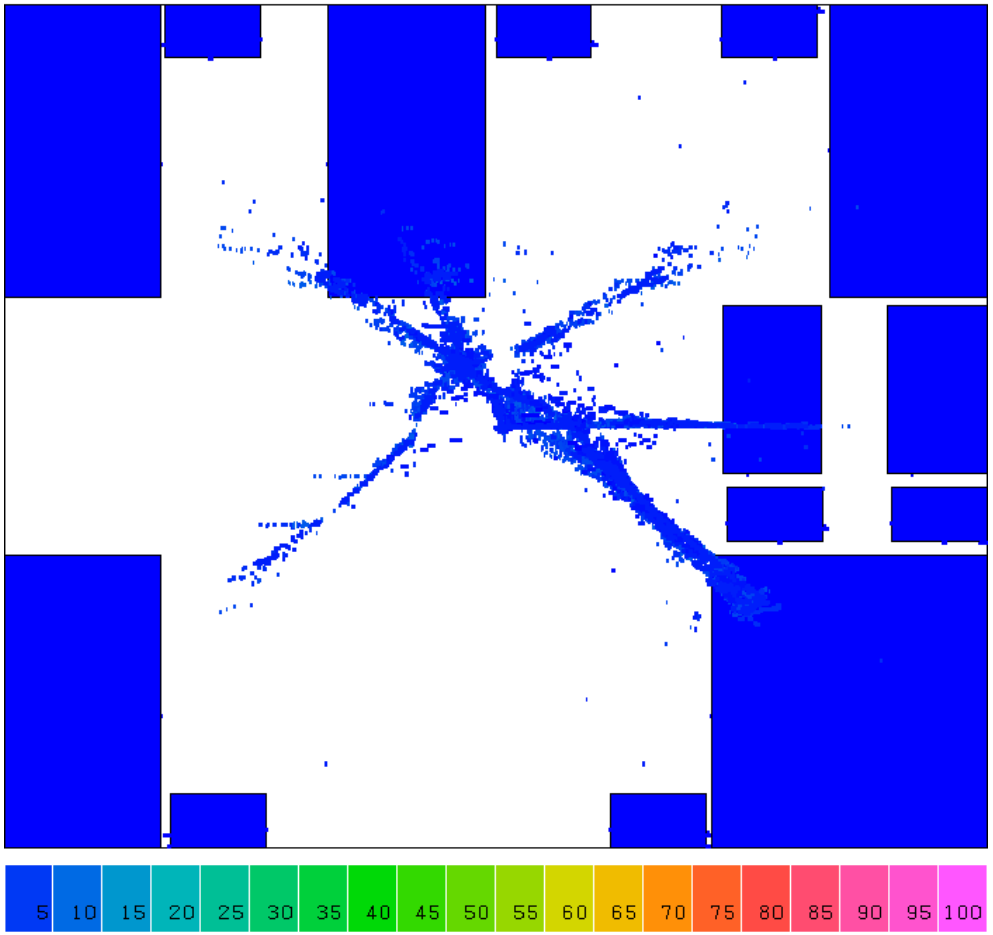


HDP
Global

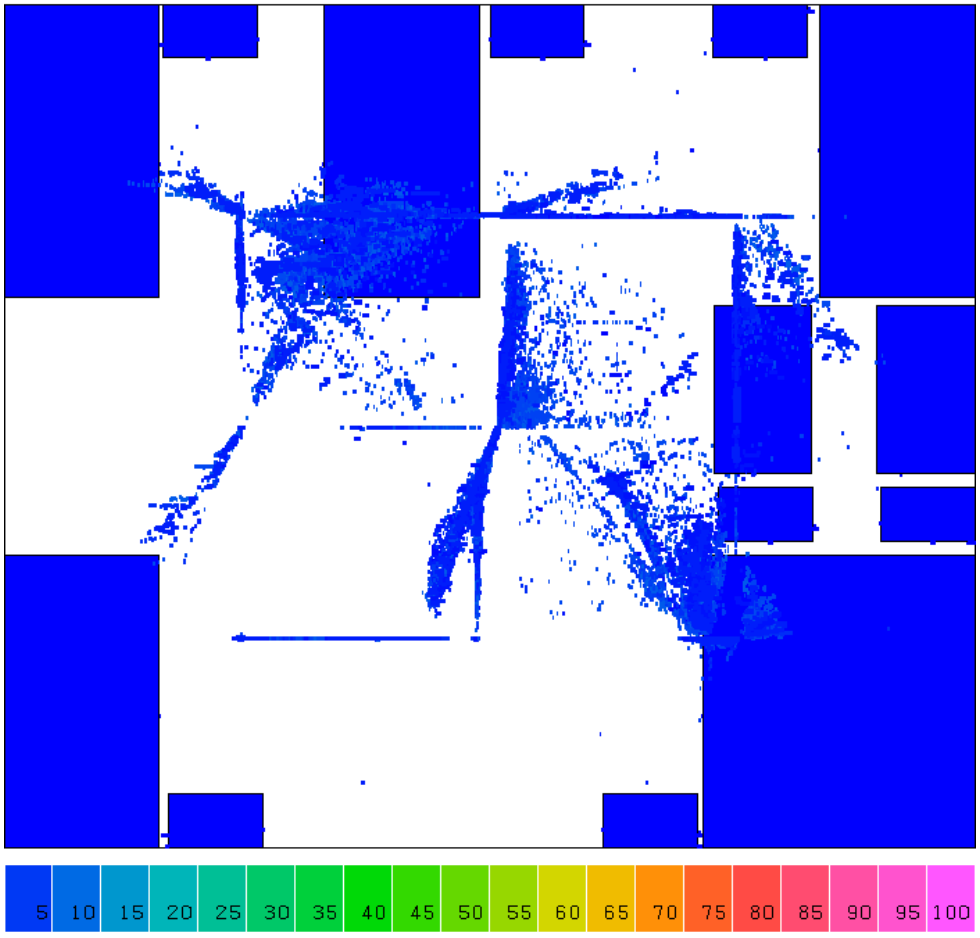
Inflation of circuits

- Used previously by Hou et al.
- Initial inflation (based on pin density)
- Given a circuit c in Region R, c is inflated by up to 100%
- The inflation is based on the congestion in R and the surrounding regions & the pin density in R
- Deflation is possible if the circuit is no longer critical

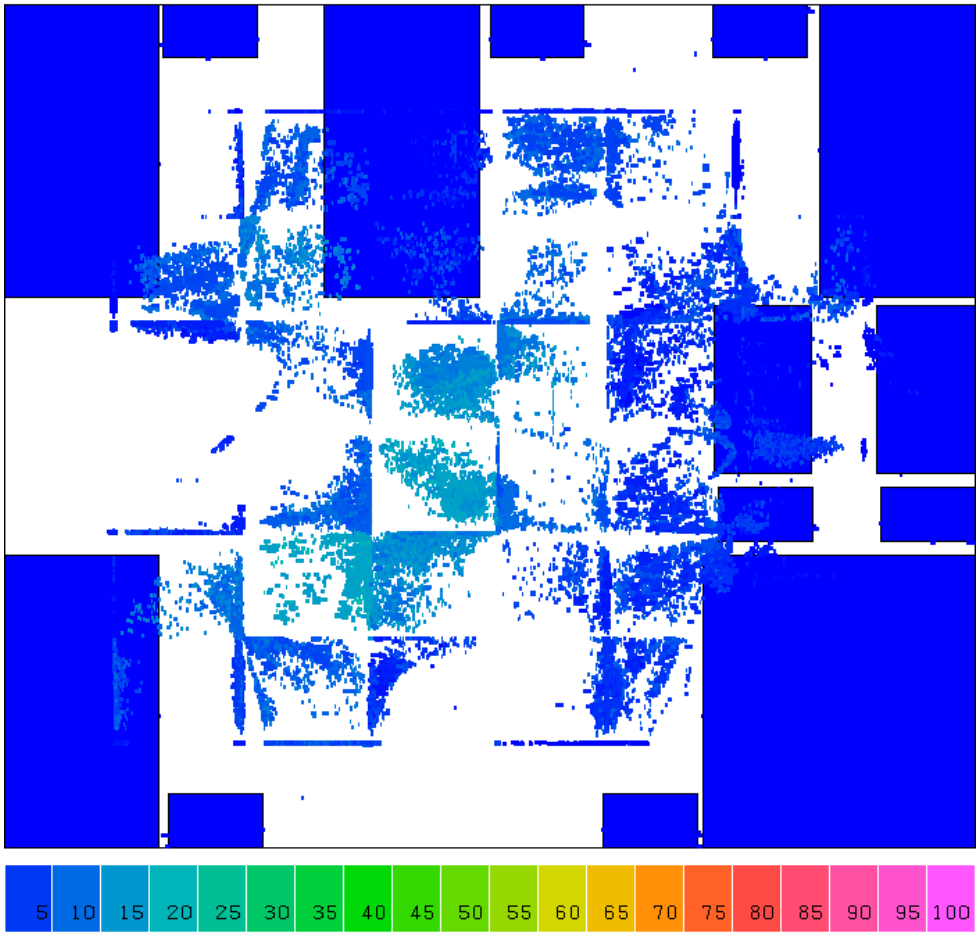
Placement Step 0



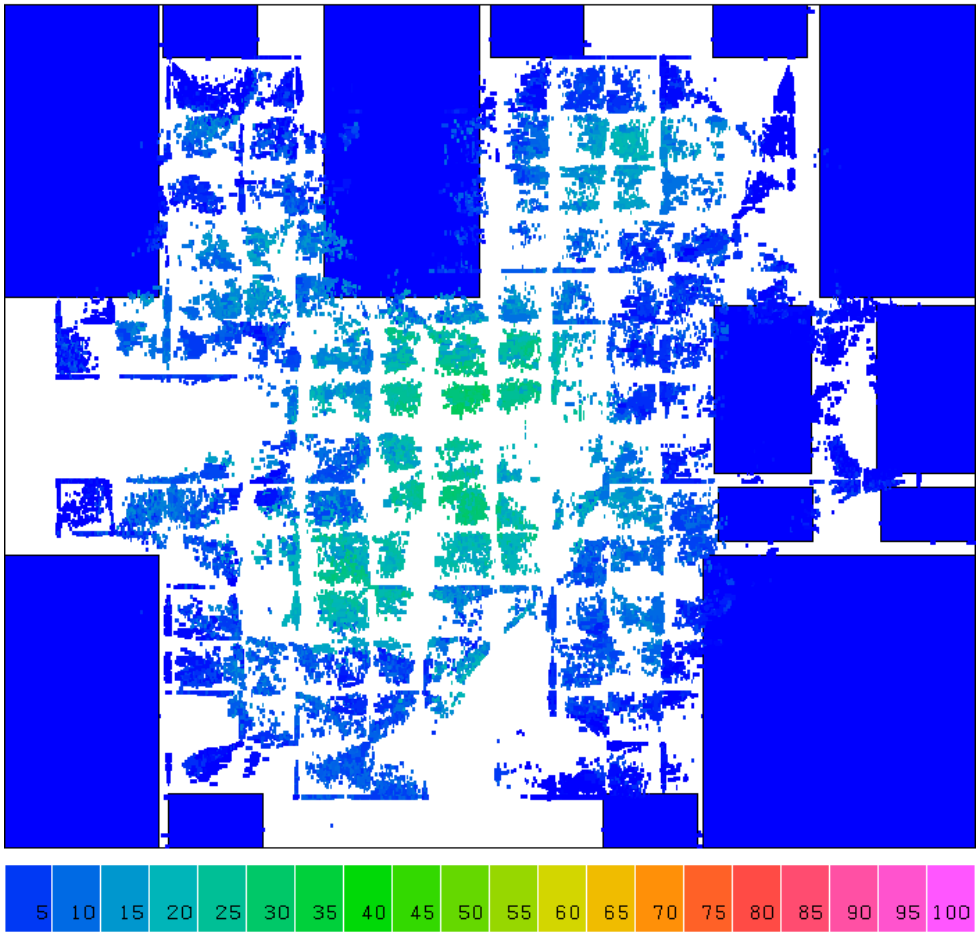
Placement Step 1



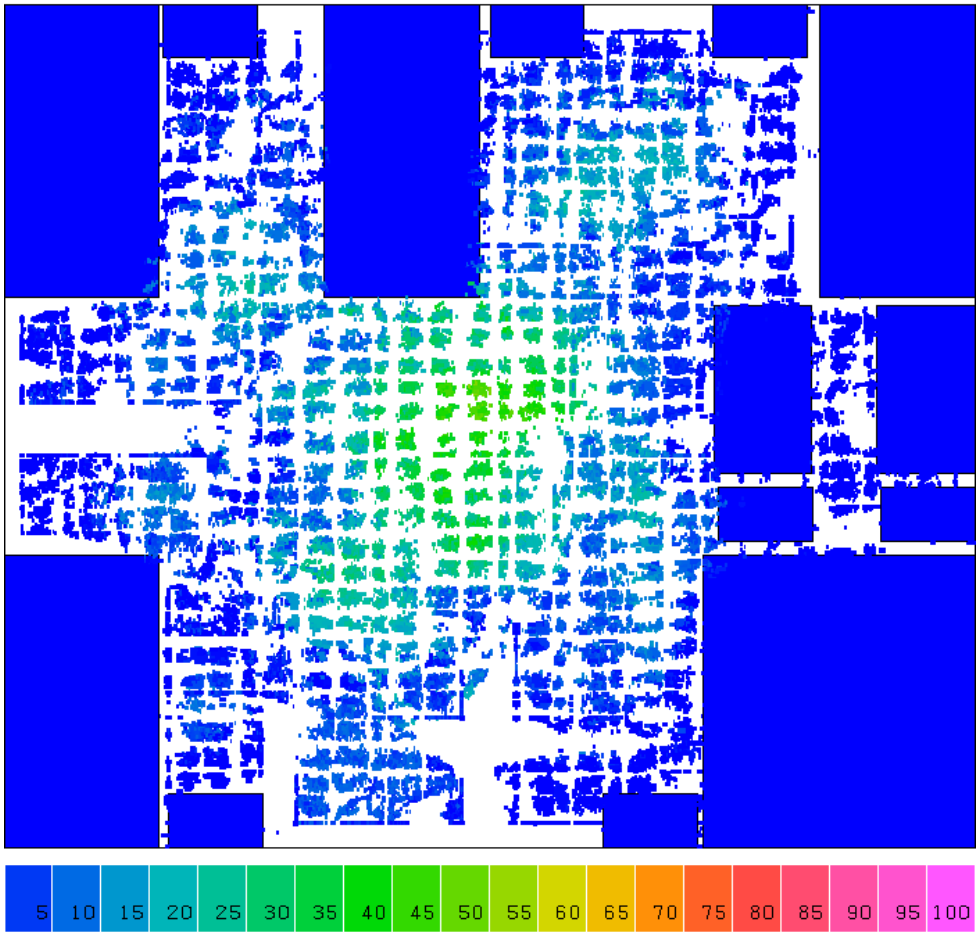
Placement Step 2



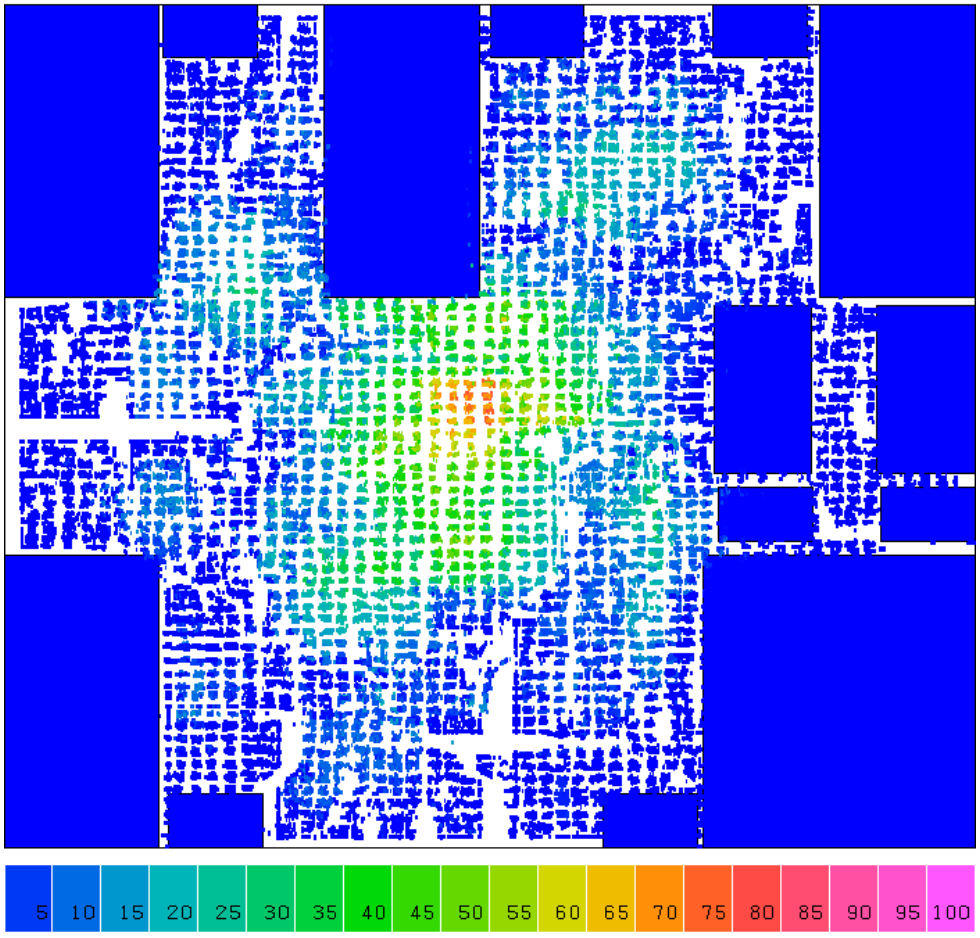
Placement Step 3



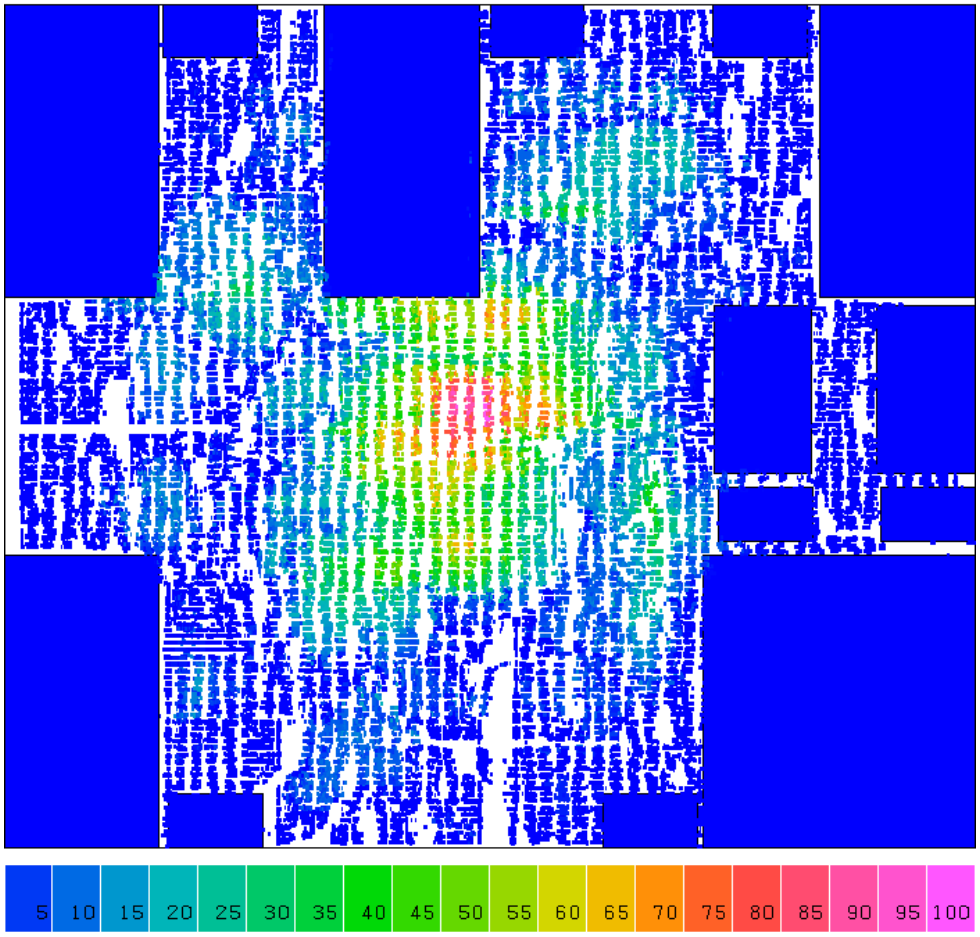
Placement Step 4



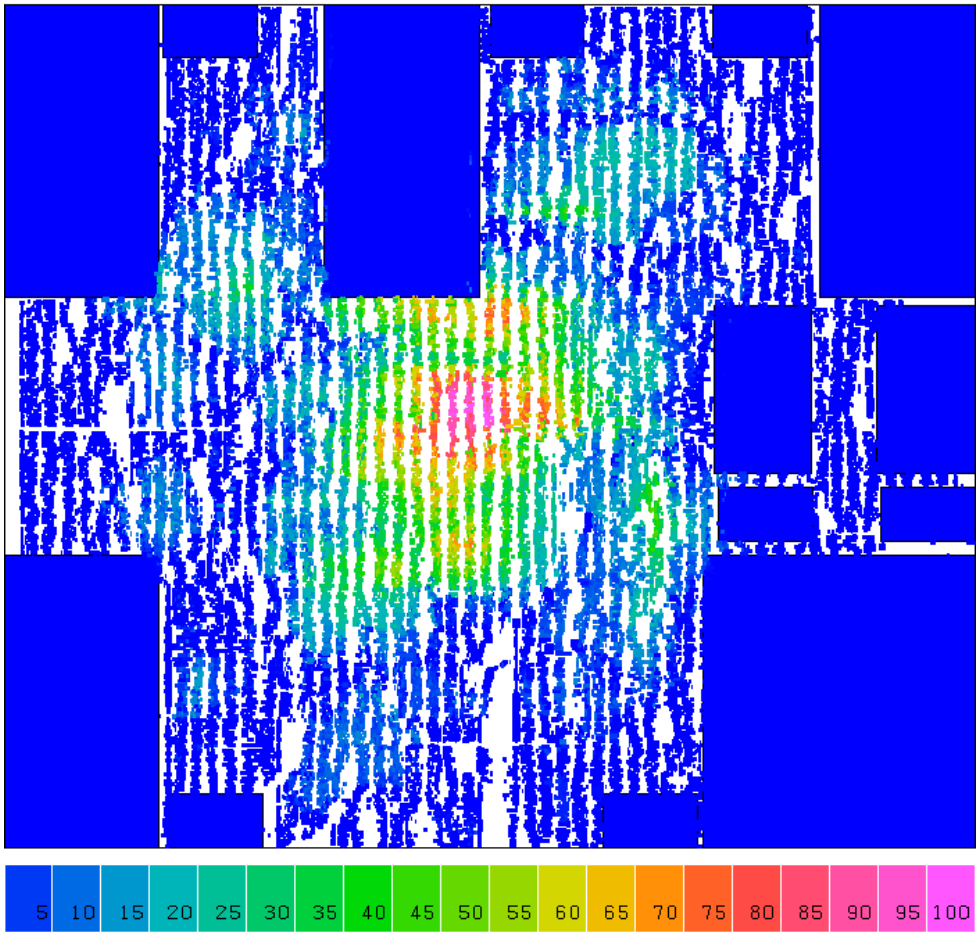
Placement Step 5



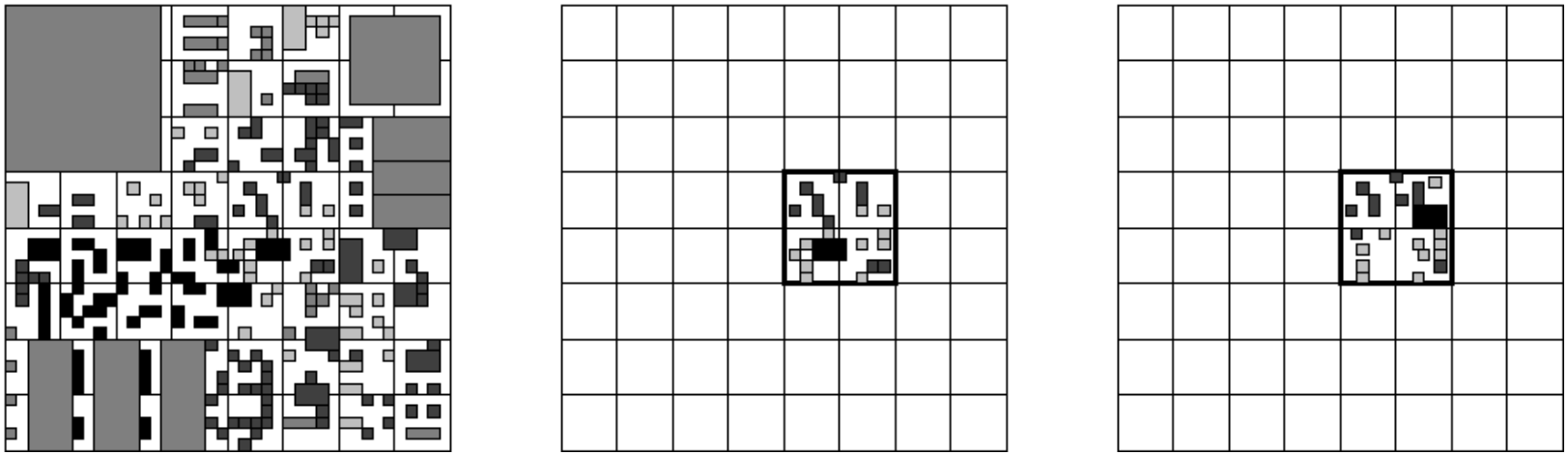
Placement Step 6



Placement Step 7



Spreading Inflated Cells



- Repartitioning considers 2x2 windows in placement grid to optimize netlength
- Use extra repartitioning step to move cells away from overloaded regions

Summary: Algorithm Overview

1. Init:
   Set window_set := {chip area}, set circuit_list(chip area):={all circuits}

2. Main Loop:
   While (window size big enough)
      Solve a QP to minimize quadratic netlength
      For (each window w in window_set)
         Quadrisection(w)

      Repartitioning
3. Legalization

Algorithm Overview

1. Init:
   Set window_set := {chip area}, set circuit_list(chip area):={all circuits}
   For (each c in {all circuits})
       Increase b(c) proportionally to |pins(c)|/size(c) # initial inflation b(c)
2. Main Loop:
   While (window size big enough)
       Solve a QP to minimize quadratic netlength
       For (each window w in window_set)
           Quadrisection(w)

       Repartitioning
3. Legalization

Algorithm Overview

1. Init:
   Set window_set := {chip area}, set circuit_list(chip area):={all circuits}
   For (each c in {all circuits})
       Increase b(c) proportionally to |pins(c)|/size(c) # initial inflation b(c)
2. Main Loop:
   While (window size big enough)
       Solve a QP to minimize quadratic netlength
       For (each window w in window_set)
           Quadrisection(w)
           Compute congestion and update b(c) # update inflation b(c)
           Quadrisection(w)

       Repartitioning
3. Legalization

Algorithm Overview

1. Init:
   Set window_set := {chip area}, set circuit_list(chip area):={all circuits}
   For (each c in {all circuits})
       Increase b(c) proportionally to |pins(c)|/size(c) # initial inflation b(c)
2. Main Loop:
   While (window size big enough)
       Solve a QP to minimize quadratic netlength
       For (each window w in window_set)
           Quadrisection(w)
           Compute congestion and update b(c) # update inflation b(c)
           Quadrisection(w)
       Reduce overloaded windows # extra repartitioning steps
       Repartitioning
3. Legalization

Computational Results

	Standard		Congestion Driven		
Chip	CPU	len	CPU	len	Blow
IBM 1	0:23 h	7.2 m	0:26 h	7.4 m	10.2 %
IBM 2	0:26 h	7.9 m	0:27 h	9.0 m	6.6 %
IBM 3	3:50 h	134 m	4:39 h	142 m	20.1 %
IBM 4	7:08 h	241 m	7:24 h	270 m	20.2 %
IBM 5	16:10 h	375 m	16:37 h	406 m	57.8 %
Mean			+8.7 %	+8.5%	

Computational Results II

	Standard				Congestion Driven			
Chip	HDP	ov	CPU	len	HDP	ov	CPU	len
IBM 1	81.7	8374	0:15 h	9 m	75.5	0	0:05 h	7.5 m
IBM 2	82.7	7000	0:19 h	11.5 m	75.4	0	0:05 h	10.1 m
IBM 3	88.8	78111	47:36 h	162 m	77.3	0	4:51 h	164 m
IBM 4	82.8	972	7:18 h	324 m	75.2	0	2:48 h	326 m
IBM 5	89.9	14382	70:57 h	512 m	84.2	0	29:48 h	527 m
Mean					-9 %		-73 %	-5.2 %

Summary

- Two important concepts during placement to consider besides wire length
  - Timing driven placement, using net-weighting
    - A new sensitivity based net weighting in ISPD'04 paper
  - Congestion minimization (using ISPD'02 as an example)
    - Congestion estimation
    - Inflate cells in congested region
    - Spread inflated cells



© 2002 National Geographic Society. All rights reserved.

NATIONALGEOGRAPHIC.COM